
MEMORIAS DEL CURSO DE PROGRAMACIÓN ORIENTADA POR OBJETOS (POO)

Gabriel Mañana Guichón

Departamento de Sistemas
Facultad de Ingeniería
Universidad Nacional de Colombia

Introducción

Este curso y el presente documento están basados en el curso de "Análisis y Diseño Orientados por Objetos (ADOO)", dictado por el mismo autor. En este curso se presentó una metodología de ADOO basada principalmente en la propuesta de Wirfs-Brock et al. en [Wirfs-Brock, 90] y en diferentes ideas planteadas por B. Meyer en [Meyer, 88] y por P. Coad y E. Yourdon en [Coad, 92]. A medida que se fueron presentando los procesos propuestos por la metodología, se desarrolló un primer diseño de un editor gráfico elemental, para ser ejecutado en un ambiente multiventanas.

El objetivo de este curso es estudiar las herramientas provistas por el lenguaje C++, en particular el compilador de Borland Int. versión 4.0. Para esto se analizan los tópicos más relevantes del lenguaje (generación de clases, herencia, sobrecarga, polimorfismo, bibliotecas de clases, streams, etc.), se hace una revisión de los aspectos más importantes a tener en cuenta en la creación de programas para Windows (recursos, eventos, tablas de respuesta, etc.), y se presenta la implementación del programa diseñado en el curso anterior.

Este documento está organizado en las siguientes secciones:

1. En la primera sección se presenta el lenguaje C++ como una extensión del lenguaje C. Se analizan tópicos conocidos tales como declaración de variables y nuevas facilidades: referencias, parámetros por omisión, operadores *new* y *delete*, etc..
2. En esta sección se estudia la sintaxis utilizada para la declaración e instanciación de clases de objetos y tópicos relacionados: control de acceso a los miembros de una clase, apuntador *this*, miembros (atributos y funciones) estáticos, funciones miembro (métodos), métodos *inline*, constructores y destructores, calificador *friend*.
3. Una vez que se tienen las nociones básicas del lenguaje se comienza el estudio del mecanismo de herencia. Se analizan la herencia simple y múltiple, conversiones implícitas de tipo, prevalencia, herencia virtual y otros tópicos relacionados.
4. En esta sección se estudia el mecanismo de sobrecarga de funciones y operadores. Sobrecarga de operadores por medio de funciones miembro y sobrecarga de operadores por medio de funciones *friend*. También se estudia un constructor especial, utilizado para la generación de copias de un objeto, llamado *copy constructor*.
5. Junto con el mecanismo de herencia, el polimorfismo es la otra herramienta en que se basa el poder de la programación orientada por objetos. Se analiza el mecanismo y se discuten

ejemplos de código tomados del programa graficador, donde se hace uso extensivo del polimorfismo

6. Borland C++ provee tres grandes bibliotecas de clases: la jerarquía de clases contenedoras (*Containers Class Library*), la jerarquía de *streams* y la jerarquía OWL (*Object Windows Library*). La jerarquía de clases contenedoras incluye las Estructuras Fundamentales (*Fundamental Data Structures*) y los Tipos Abstractos (*Abstract Data Types*). Aquí se estudia los conceptos de clase genérica y de clase contenedora, y se muestran partes de código tomados también del programa de ejemplo. Por su importancia, la jerarquía de *streams* y OWL serán estudiadas en las secciones 7 y 9 respectivamente.
7. C++ presenta una nueva técnica para el manejo de entrada/salida: *streams*. Los *streams* de C++ proveen la funcionalidad de la biblioteca *stdio* (*standard input/output*) del lenguaje C, más un conjunto de nuevas facilidades. Se aborda temas como entrada/salida estándar, manipuladores, manejo de archivos y formato de cadenas de caracteres en memoria.
8. Los objetos de interfaz tales como ventanas, cajas de diálogo y demás, o los objetos que una aplicación manipula, como los ObjetosGráficos en el caso de nuestro ejemplo, son creados en tiempo de ejecución y destruidos al terminar el programa. Estos objetos pueden ser creados con la capacidad de ser transportables (*streamable*) y por lo tanto escritos en (transportados a) disco. De esta manera se dice que los objetos persisten a la aplicación, o de otra forma, que son *persistentes*. En esta sección se estudian las macros y clases provistas por Borland C++ 4.0 para la construcción de clases persistentes, y se muestra, como ejemplo, la implementación de la persistencia de los ObjetosGráficos que el programa de ejemplo manipula.
9. En esta última sección se estudian los tópicos más relevantes para la generación de programas que han de ser ejecutados en ambiente MS-Windows. Se presenta el proceso de generación y utilización de recursos, se analiza el tipo de multitarea soportado por Windows y se analiza la programación orientada al procesamiento de eventos o mensajes, junto con las macros y clases provistas por la biblioteca OWL (*ObjectWindows Library*) para este fin. Como culminación del curso se presenta el código completo del graficador implementado, donde se abordan todos los tópicos estudiados.

1. C++, una extensión del lenguaje C

El lenguaje C++ se puede considerar como una extensión del lenguaje C. Un programa escrito en C++ está compuesto por clases. A partir de estas clases se han de crear (en tiempo de ejecución) los objetos que proveen la funcionalidad de la aplicación. Las clases a su vez (esto se analizará en detalle en la sección 2) están compuestas de atributos (variables) y de métodos (funciones). Salvo ciertas extensiones que ofrece el C++, los procesos de declaración de variables e implementación de funciones siguen siendo iguales. Las funciones miembro de las clases (métodos) pueden retornar un valor o no (*void*), y siguen recibiendo parámetros en la misma forma: por valor o por referencia. En esta sección se presentan las nuevas herramientas que provee el lenguaje C++ y se asume el conocimiento de los conceptos básicos para la programación en lenguaje C.

1.1 Comentarios

Se siguen utilizando las parejas de `/* ... */` para comentar bloques y se pueden utilizar dos oblicuas `//` para comentar hasta el final de la línea.

```

/*
    comentario
    multilínea
    igual que en C
*/

// nueva forma:
// comenta hasta el fin de la línea

    :

    ObjetoGrafico::display( tdc );    // llama a la función display()
                                     // de la clase ancestra
}    // Elipse::display

```

1.2 Variables

Las variables siguen caracterizadas según los tres factores estándares:

- vida (*scope*)
- tipo (*type*)
- clase de almacenamiento (*storage class*)

i. vida o alcance

Según esta clasificación existen variables locales y variables globales. Las variables locales o automáticas 'nacen' al comienzo del bloque o función donde son declaradas y 'mueren' al terminar el bloque o función y por lo tanto sólo pueden ser accesadas dentro del bloque o función en cuestión. Deben ser inicializadas explícitamente ya que el compilador no lo hace. Dado que 'mueren' al terminar la función, no se puede asumir que al ejecutar la función nuevamente las variables locales tengan el último valor asignado. Si esto es lo que se necesita, se agrega el calificador **static** a la declaración de la variable. A diferencia del lenguaje C, el C++ permite la declaración de variables en cualquier parte de la función.

```
{      // comienzo de bloque

      // final
}
```

```
void f()
{
    int a;          // local a f(), tiene cualquier valor
    static int b;    // local a f(), inicializada en 0 por el compilador, preserva su
                    // valor entre llamadas a f()
    :

    for ( int c = 0; c < 512; c++ )
    {
        int d;      // local al for, no inicializada por el compilador
        d = ...;    // bien
    }

    a = 0; // bien
    b = 0; // bien
    c = 0; // bien
    d = 0; // error

    int d; // bien
    :
}

void g()
{
    a = 0; // error
    b = 0; // error
    :
}
```

Las variables globales son declaradas (por lo general al comienzo) por fuera del cuerpo de las funciones del programa. Tienen alcance de archivo, esto es, pueden ser accesadas por cualquier función dentro del módulo .c donde son declaradas (encadenamiento interno). Se declaran como **extern** (fuerza encadenamiento externo) en los otros módulos donde se necesite tener acceso a ellas. Son inicializadas en cero por el compilador.

```
int a;    // inicializada en 0 por el compilador

void f()
{
    a = ...; // bien
    :
}
```

ii. tipos

Está también la diferenciación de variables según el tipo de dato:

tipos intrínsecos

char	(1 byte)
short	(2 bytes)
int	(2 bytes)
long	(4 bytes)
float	(4 bytes)
double	(8 bytes)
long double	(10 bytes)
apuntadores	(near 2b, far 4b)

definidos por el usuario (UDT)

arreglos	
estructuras	(struct)
uniones	(union)

iii. clase de almacenamiento

Existen calificadores que permiten definir la clase de almacenamiento que se utilizará para la variable:

```

auto int i;           // variable automática, calificador por omisión
register int r;        // utiliza un registro del procesador
extern int e;          // declarada en otro módulo
static int s;          // preserva el contenido

```

y modificadores tales como:

```

const int i = 0;       // de solo lectura, debe ser inicializada
volatile char c;        // el compilador no debe asumir que la variable conserva
                        // su valor entre instrucciones (al optimizar por ejemplo),
                        // puede ser modificada por una ISR

// el uso de 'variables' const es más apropiado que el uso de constantes
// declaradas con #define, ya que el compilador puede verificar los tipos
// involucrados en la operación

const double PI = 3.14159; // no utiliza memoria
:
int i;
:
i = PI;                  // warning

void print( const char* szMsg )
{
    // szMsg no puede ser modificado, no efectos secundarios
}

```

1.3 Nuevas facilidades en C++

Aparece una evolución del tipo apuntador: *referencias*. Al igual que los apuntadores guardan la dirección de la variable a la cual referencian, pero permiten una notación más uniforme y además deben ser inicializados al declararlos, con lo que se evitan los problemas causados por los apuntadores 'salvajes' o no inicializados. La notación es la siguiente:

```

int a;
int& r;                // error
int& r = &a;           // referencia a un entero, almacena la dirección de a
r = 0;                 // a = 0

void f( int& x )

```

```

{
    x = ...;      // se utiliza como un argumento pasado por valor
    :
}

void g()
{
    int x;
    :
    f( &x );      // se pasa la dirección
}

```

Parámetros por omisión: el compilador puede asignar valores por omisión a los parámetros de una función que sean omitidos. Sólo se puede aplicar a los últimos parámetros. Su utilización es como sigue:

```

void MoveWindow( HWND hwnd, BOOL bEraseBackg = FALSE )
{
    :
}

MoveWindow( hw );      // bien
MoveWindow( hw, TRUE ); // bien

void f( int a = 0, int b = 0 )      // bien
{
    :
}

void g( int a, int b = 0 , int c = 1 )      // bien, a obligatorio, b y c opcionales
{
    :
}

void h( int a = 0, int b, int c = 1 )// error
{
    :
}

```

Una de las ventajas del C++ frente a otros lenguajes orientados por objetos radica en que es un lenguaje híbrido. Lo anterior significa que se pueden utilizar funciones en C estándar mezcladas en

código orientado por objetos (clases). Para indicarle al compilador que se trata de funciones en C, el prototipo de estas funciones debe ser declarado de la siguiente forma:

```
extern "C" void putc( const int __c, FILE _FAR* __fp );

// o también:
extern "C" {
    // prototipos de funciones
    :
    :
}
```

Además del manejo estándar de memoria dinámica a través de las funciones como malloc(...), calloc(...), free(...) y otras, ahora existen dos operadores más flexibles (pueden ser sobrecargados, esto se trata más adelante): *new* y *delete*.

```
// Ejemplos:

int* v = new int[ 1024 ];           // pide 2048 bytes
TWindow* pWin = new TWindow(...);   // pide memoria para un objeto de la clase
                                     // TWindow y lo construye
:
delete [] v;                         // libera memoria
delete pWin;                         // destruye el objeto y libera memoria ocupada
```

2. Clases

El enfoque orientado por objetos aborda el manejo de la complejidad de todo sistema en el mundo real, haciendo una abstracción de su conocimiento y encapsulándolo en objetos. Acostumbramos dividir la información en datos y funciones. Este enfoque, llamado procedimental o descomposición funcional, aborda primero las tareas a llevar cabo y las va descomponiendo en sub-tareas (funciones o procedimientos) hasta llegar al nivel de instrucciones directamente codificables en el lenguaje de programación. El enfoque orientado por objetos descompone el sistema en un conjunto de entidades que "saben jugar su papel" dentro del sistema. Cada entidad es responsable de una parte del conocimiento total del sistema: debe ser capaz de 'recordar' ciertos datos y debe ser capaz de 'realizar' ciertas operaciones. Esta práctica de agrupar datos y procedimientos (operaciones sobre los datos) en una entidad es conocida como *encapsulamiento*. El encapsulamiento ayuda en el manejo de la complejidad inherente de todo sistema particionando y distribuyendo esta complejidad en entidades separadas: objetos.

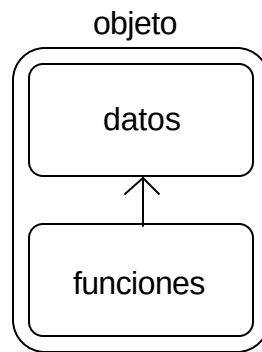


Fig. 1. Encapsulamiento.

Los objetos que comparten un mismo conjunto de atributos y un mismo comportamiento pertenecen a una misma *clase*. De esta forma, una clase es la especificación genérica de un conjunto arbitrario de objetos similares. Los objetos creados a partir de una clase en particular son llamados *instancias* de esa clase. El concepto de clase (a nivel sintáctico) puede ser asimilado como la evolución del concepto de tipo dentro de un contexto clásico, así como los objetos pueden ser asimilados como una evolución de las variables.

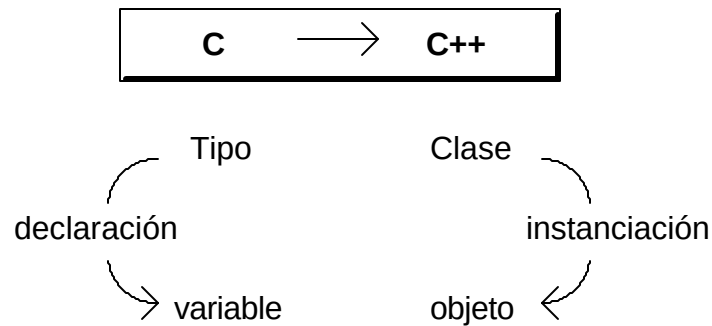


Fig. 2. Clase: evolución del concepto de tipo.

En un lenguaje estructurado, como C, se 'declaran' variables de cierto tipo. En un lenguaje orientado por objetos, un objeto es una 'instanciación' de una clase en particular. El conjunto de datos que un objeto encapsula (atributos del objeto) es referido como el 'conocimiento' que el objeto posee, y el conjunto de operaciones que es capaz de realizar (métodos) se conoce como su 'comportamiento'.

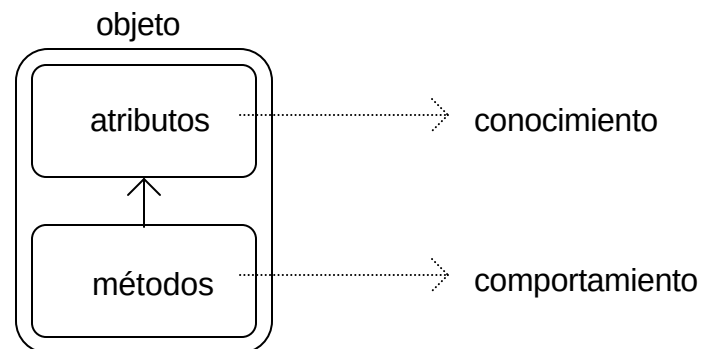


Fig. 3. Conocimiento y comportamiento.

2.1 Interfaz pública

Durante la ejecución de un programa se crean objetos, se agrupan (eventualmente), y se destruyen. De esta forma, en un ambiente orientado por objetos, un programa está compuesto por un conjunto de objetos que interactúan para llevar a cabo una determinada tarea: la tarea para la cual se ha desarrollado el programa. Esta interacción entre objetos se da a través de mensajes. Todo objeto tiene una faceta privada (el conocimiento y comportamiento que encapsula) y una interfaz pública por medio de la cual se comunica con los demás objetos del sistema. Este esquema de comunicación ha sido modelado como una relación cliente-servidor. El objeto que en determinado momento asume el rol de cliente envía un mensaje al objeto que actúa como servidor y este responde en forma apropiada: retornando alguna información o realizando

alguna tarea. Existe entonces una especie de protocolo de comunicación para cada clase de objetos de la aplicación. Los mensajes permitidos dentro del protocolo de comunicación con una clase están representados por los métodos que hacen parte de la interfaz pública de la clase.

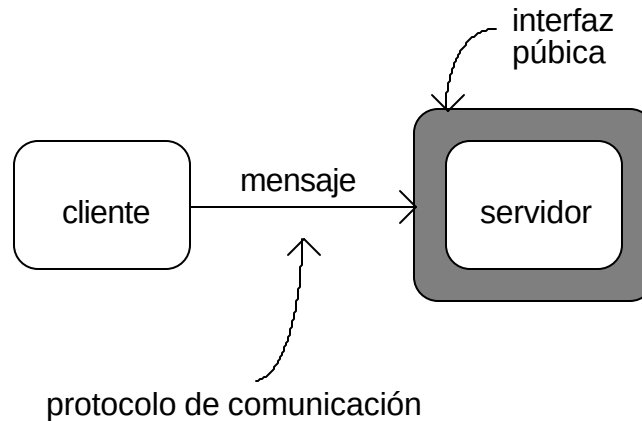


Fig. 4. Interacción entre objetos.

2.2 Declaración de clases e instanciación

A continuación se muestra la sintaxis utilizada para la declaración de una clase y la instanciación de objetos de esa clase.

```

class <nombre de la clase>
{
    // atributos
    :

    // métodos
    :
};

// Ejemplo:

// declaración de la clase
class Punto
{
    int x;                // declaración de atributos
    int y;

public:
    void posición( int nx, int ny ); // declaración de funciones miembro
    void mover( int dx, int dy );
    :
};
  
```

```

// definición de la clase = definición de funciones miembro
//
// nota: el operador :: (scope resolutor) determina la clase
// a la cual pertenece la función que se está definiendo

void
Punto::posicion( int nx, int ny )
{
    x = nx;
    y = ny;
}

void
Punto::mover( int dx, int dy )
{
    x += dx;
    y += dy;
}

// ejemplo de instanciación

void main()
{
    Punto p;                // declaración de un objeto de la clase punto

    // se utiliza el operador . para referenciar componentes del objeto,
    // al igual que en C para referenciar un campo de una estructura

    p.posicion( 0, 0 );      // bien, posición() hace parte de la interfaz pública
    p.mover( 5, 10 );        // bien, idem

    p.x = 0;                 // error, x es un miembro privado
}

```

2.3 Control de acceso

Hemos visto que una clase tiene componentes privados y componentes públicos. Ahora veremos la sintaxis utilizada para controlar el acceso a los componentes de una clase.

Existen tres tipos de 'usuarios' de los componentes de una clase:

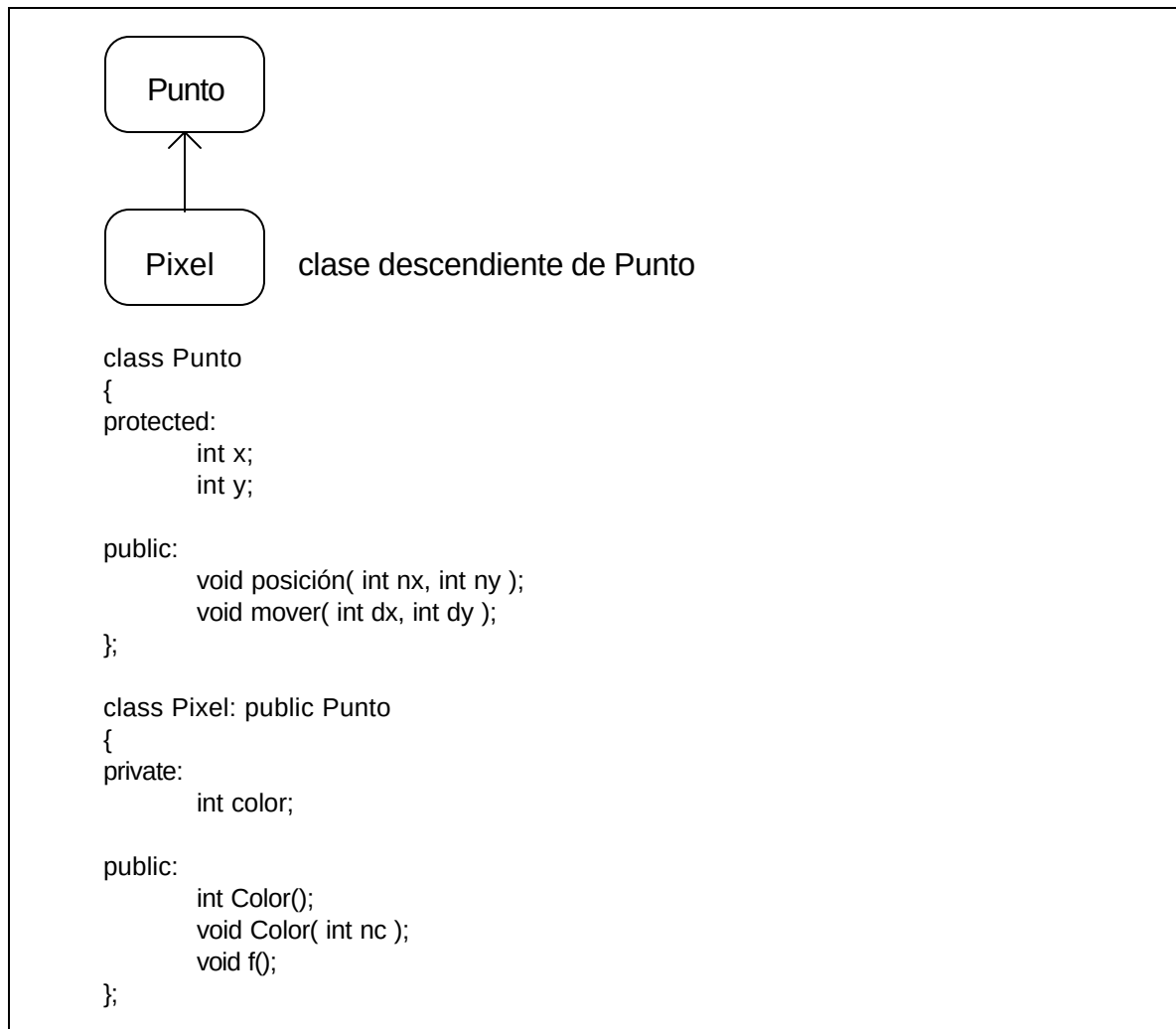
- la propia clase (objetos de esa clase)
- clases derivadas (herencia, objetos de clases descendientes)
- otras clases de la aplicación

Para diferenciar el acceso de los diferentes usuarios se utilizan tres palabras reservadas:

private: sólo la clase (y funciones y clases *friend*: esto más adelante) tiene acceso a los miembros privados; este el calificativo por omisión y se utiliza para los atributos (sino se estaría yendo en contra del principio de encapsulamiento) y métodos internos.

protected: sólo la clase y clases derivadas (herencia) tienen acceso a los miembros protegidos.

public: todas las clases tienen acceso a los miembros públicos; utilizado únicamente para los métodos que hacen parte de la llamada interfaz pública.



```

int
Pixel::Color()          // método consultor
{
    return color;
}

void
Pixel::Color( int nc )   // método modificador
{
    color = nc;
}

void
Pixel::f()
{
    x = 0;  // correcto sintácticamente
    y = 0;  // idem
    :
};

// sin embargo sería mejor:

class Punto
{
private:
    int x;
    int y;
public:
    int X() { return x; }          // consultor
    int Y() { return y; }          // consultor
    void posicion( int nx, int ny ); // modificador
    void mover( int dx, int dy );   // modificador
};
:

// entonces:

void
Pixel::f()
{
    if ( X() < 0 )
    {
        posicion( 0, Y() );
    }
    :
}

```

2.4 Apuntador *this*

Toda función miembro de una clase (que no sea calificada **static**) recibe implícitamente un apuntador al objeto que la invoca con el fin de tener acceso a los atributos del mismo: este apuntador se llama *this*.

```
// Ejemplo:

void
Punto::posicion( int nx, int ny )
{
    x = nx;        // this→x = nx;
    y = ny;        // this→y = ny;
}

// por lo tanto, si desde un método de una clase se necesita retornar
// una referencia al objeto en tiempo de ejecución:

void
Pixel::f()
{
    :
    return this;
}
```

Se debe recordar que código se escribe para las clases pero será utilizado por los objetos en tiempo de ejecución.

2.5 Miembros estáticos (*static*)

```
// atributos

class Ejemplo
{
private:
    static long cantidad;    // declaración, existe solo un atributo 'cantidad' para
                           // todos los objetos de la clase Punto y existe siempre,
                           // se creen objetos de la clase Punto o no.
```

```

};      :

// debe ser definido sino da un error de encadenamiento

long Ejemplo::cantidad = 0;
      :

// métodos

class Ejemplo
{
    static long cantidad;

public:
    static long Cantidad(); // a diferencia de los métodos comunes, los métodos
                          // estáticos no reciben apuntador this y por lo tanto sólo
                          // pueden referenciar miembros (atributos o funciones)
                          // estáticos
      :
};

long
Ejemplo::Cantidad()
{
    return cantidad;      // bien
}

```

Los miembros estáticos de una clase se pueden utilizar sin referenciar un objeto en particular, usando el nombre de la clase y el operador `::` (scope resolutor):

```

void main()
{
    long l = Ejemplo::Cantidad();    // bien
    :
}

```

2.6 Funciones miembro

Existen diferentes tipos de funciones miembro (métodos) de una clase. A continuación se hace una revisión de éstos.

2.6.1 Funciones *const*

Retornan un objeto constante, esto es, el objeto retornado por una función *const* no puede ser modificado.

```
class Ejemplo
{
```

```
    public:
        const char* nombre();    // retorna una cadena de caracteres que no puede ser
                                // modificada
    };

    const          // hay que incluirlo en la definición
    char*
    Ejemplo::f()
    {
        return "Ejemplo";
    }

    void main()
    {
        Ejemplo e;

        printf( "%s", e.nombre() );    // bien
        strcpy( e.nombre(), "xxx" );    // error
    }
```

2.6.2 Funciones que reciben el apuntador *this* constante

Pueden acceder los atributos de la clase pero no modificarlos.

```
class Ejemplo
{
    private:
        int valor;

    public:
        int Valor( ) const;
        :
};

int
```

```

Ejemplo::Valor() const // hay que incluirlo en la definición
    valor = ... ;      // error

    return valor;

}

```

```

// utilizadas en funciones que simplemente retornan valores, y
// utilizadas en superclases para evitar que funciones miembro de clases derivadas
// (que redefinan su comportamiento) no puedan modificar los atributos de la clase
// ancestra

```

2.6.3 Funciones *inline*

Son expandidas en donde son evocadas, al igual que las macros en C. Al igual que las macros también, agilizan la ejecución del programa pero lo hacen más grande. El calificador *inline* se utiliza en funciones breves, que solamente retornan un valor o que simplemente llaman a otra función. Si la función es grande, en cuanto al código que genera, el compilador no la expande *inline* aunque así haya sido calificada.

```

class Ejemplo
{
private:
    int valor;
public:
    int Valor() { return valor; } // inline
};

// o mejor

class Ejemplo
{
private:
    int valor;
public:
    int Valor();
};

inline
int
Ejemplo::Valor()
{
    return valor;
}

```

2.6.4 Constructores

Utilizados para construir un objeto de una clase determinada. Si no se declara ninguno, el compilador asigna uno por omisión. Asimismo, se puede declarar uno ó más, siempre con nombre igual al nombre de la clase. Los constructores no pueden retornar un valor.

Diferentes tipos:

```
// a. constructores para inicialización por omisión

class Punto
{
private:
    Punto() { x = -1; y = -1; }      // por omisión: no recibe parámetros
                                    // nombre = nombre clase
                                    // no puede retornar un valor
};

// b. Constructores para inicialización específica

class Punto
{
private:
    int x;
    int y;
public:
    Punto() { x = -1 ; y = -1 }      // no puede retornar un valor
    Punto( int nx, int ny ) { x = nx; y = ny; } // no puede retornar un valor
    :
};

// se puede declarar todos los que sean necesarios, siempre que reciban diferente //
cantidad o tipo de argumentos (sobrecarga de funciones, más adelante)

class Punto
{
private:
    int x;
    int y;
public:
    Punto() { x = -1; y = - 1; }
    Punto( int nx, int ny ) { x = nx; y = ny; }
    Punto( float fx, float fy ) { x = fx; y = fy; }
};

// ejemplo de uso:
```

```

void main()
{
    Punto p;
    p.posicion( 0, 0 );

    Punto q( 0, 0 );
    Punto r( 1.5, 1.5 );
}

```

// c. constructores para copiar objetos (copy constructors)

```

class Punto
{
private:
    int x;
    int y;

public:
    :
    Punto( Punto& r ) { x = r.x; y = r.y; }
};

// -----
void main()
{
    Punto p( 0, 0 );
    Punto q = p;    // el compilador llama al constructor de copia
}

```

Cuando se construye un objeto de una clase descendiente se debe llamar al constructor de la clase ancestral explícitamente, antes del cuerpo del constructor.

```

class Pixel : public Punto
{
private:
    int color;
public:
    Pixel( int nx, int ny, int ncolor );
    :
};

Pixel::Pixel( int nx, int ny, int ncolor ) : Punto( nx, ny )
{

```

```
}    color = ncolor;
```

2.6.5 Destructores

Se utilizan para llevar a cabo operaciones de 'limpieza' cuando el objeto ya no es necesario: liberar memoria pedida en el constructor, liberar recursos, etc. Llevan el mismo nombre de la clase antecedido por ~. Al igual que los constructores, los destructores no pueden retornar un valor.

```
class Ejemplo
{
private:
    char* szNombre;
public:
    Ejemplo( char* sz );
    ~Ejemplo();
    :
};

Ejemplo::Ejemplo( char* sz )
{
    szNombre = new char [ strlen(sz)+1 ]; // pide memoria
    strcpy( szNombre, sz );
}

Ejemplo::~~Ejemplo()
{
    delete [] szNombre; // libera memoria
}
```

Otras características de los destructores:

1. Los destructores pueden ser virtuales (sobrecarga de funciones, más adelante), los constructores no.
2. Los destructores no pueden recibir argumentos.
3. No pueden ser llamados explícitamente: son llamados automáticamente cuando el objeto sale de alcance.

```
void f()
{
```

```

    Ejemplo e( "automático" );

} // destructor de e llamado al terminar la función

```

2.7 Calificador *friend*

Se utiliza para dar a una función o clase, acceso irrestricto a los miembros de otra clase. Se utiliza generalmente en clases relacionadas conceptualmente.

```

class Lista;                // forward declaration

class Nodo
{
    friend class Lista;      // declara Lista clase amiga de Nodo
                             // Lista tiene acceso a todos los miembros
                             // de Nodo: públicos, protegidos y privados
                             // (no importa en que sección se coloca)

private:
    :
    :

};

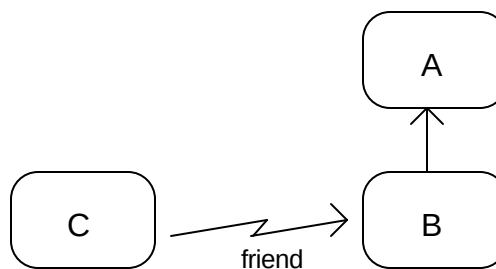
class Lista
{
private:
    Nodo* pCabeza;
    Nodo* pCorriente;
    Nodo* pUltimo;

public:
    Lista ( );
    ~Lista ( );
    agregar( Nodo* pNode );

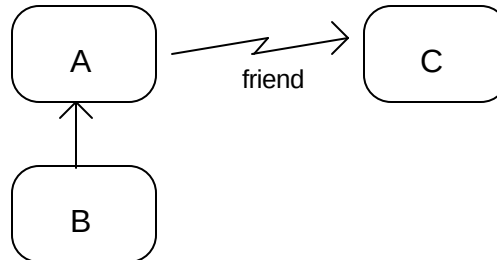
};

```

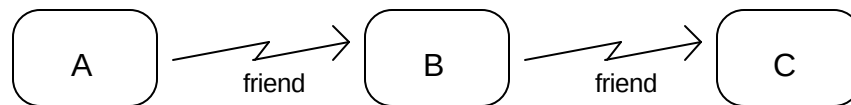
2.7.1 Propiedades de las clases *friend*



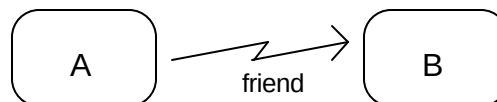
La clase C también tiene acceso a todos los miembros de A, ya que la clase A es parte de clase B.



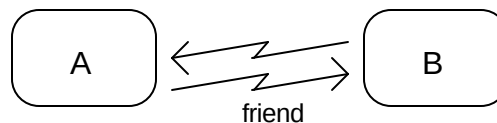
La clase B no es *friend* de la clase C, la amistad no se hereda. (igual que en el mundo real ?)



La clase A no es *friend* de la clase C. La amistad no es transitiva.



La clase B no es *friend* de la clase A. La amistad no es conmutativa. Sin embargo, si se necesita que cada clase tenga acceso a los miembros de la otra, se puede hacer lo siguiente:



3. Herencia

El mecanismo de herencia es una de las herramientas más poderosas del C++. Permite la generación de nuevas clases a partir de clases existentes, como una extensión o especialización de éstas, promoviendo de esta forma la *reutilización* de código. Cuando una clase es derivada de otra, esta hereda todos los atributos y métodos de la clase ancestral o superclase. Para la construcción de un programa entonces, se pueden utilizar las clases que vienen con el compilador y que son comunes a todo programa, ventanas, listas, etc., y desarrollar solamente aquellas que son particulares a la aplicación. Con esto el tiempo de desarrollo disminuye notablemente, y sobretodo, de esta forma se están construyendo nuevas clases que se agregan a la biblioteca existente, y que serán reutilizadas en proyectos futuros.

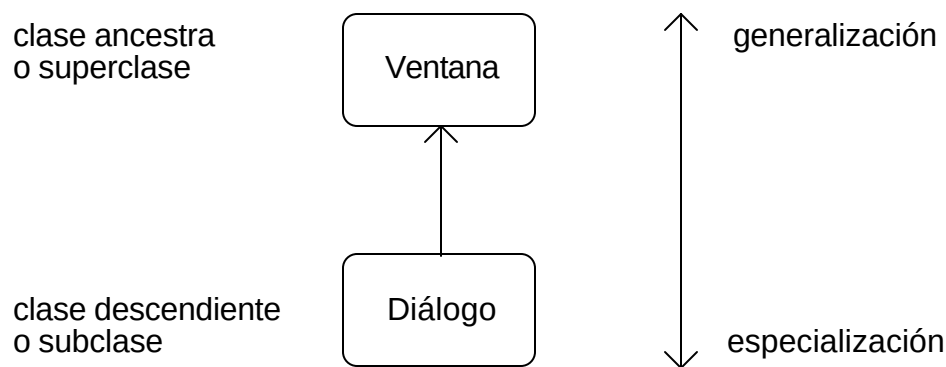


Fig. 1. Especialización a través de la herencia

El lenguaje C++ provee dos mecanismos de herencia: herencia simple y herencia múltiple.

3.1 Herencia simple y herencia múltiple

En la herencia simple una superclase puede tener n subclases, pero cada subclase sólo puede ser descendiente de una superclase.

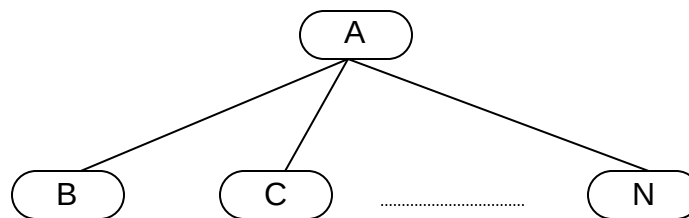


Fig. 2. Jerarquía de herencia simple

En la herencia múltiple una superclase puede tener n subclases y una subclase puede ser descendiente de una o más superclases, heredando los métodos y atributos de todas las superclases.

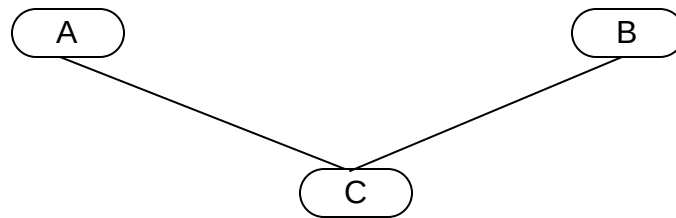


Fig. 3. Jerarquía de herencia múltiple

```

class Punto
{
private:
    int x;
    int y;
public:
    Punto( int nx, int ny );
    void Posicion( int nx, int ny );
};

// definición de los métodos de la clase Punto
:

class Pixel : public Punto
{
private:
    int color;
public:
    Pixel( int nx, int ny, int ncolor );
    void Color( int ncolor );
    int Color();
};

// definición de los métodos de la clase Pixel
:

// Ejemplo:

void main()
{
    Pixel p( 0, 0, 0 );
    p.Posicion( 10,10 );    // método heredado
    p.Color( 5 );           // notación invariante: no se hace diferencia entre
                           // métodos propios o heredados
}
  
```

Si la clase Punto ya ha sido depurada, entonces sólo se debe depurar los métodos que Pixel agrega. Las características (A/M) heredadas pueden ser extendidas, modificadas, restringidas,

eliminadas o utilizadas tal cual. De esta manera se hace un desarrollo incremental de la aplicación, de una forma más flexible que en el enfoque estructurado.

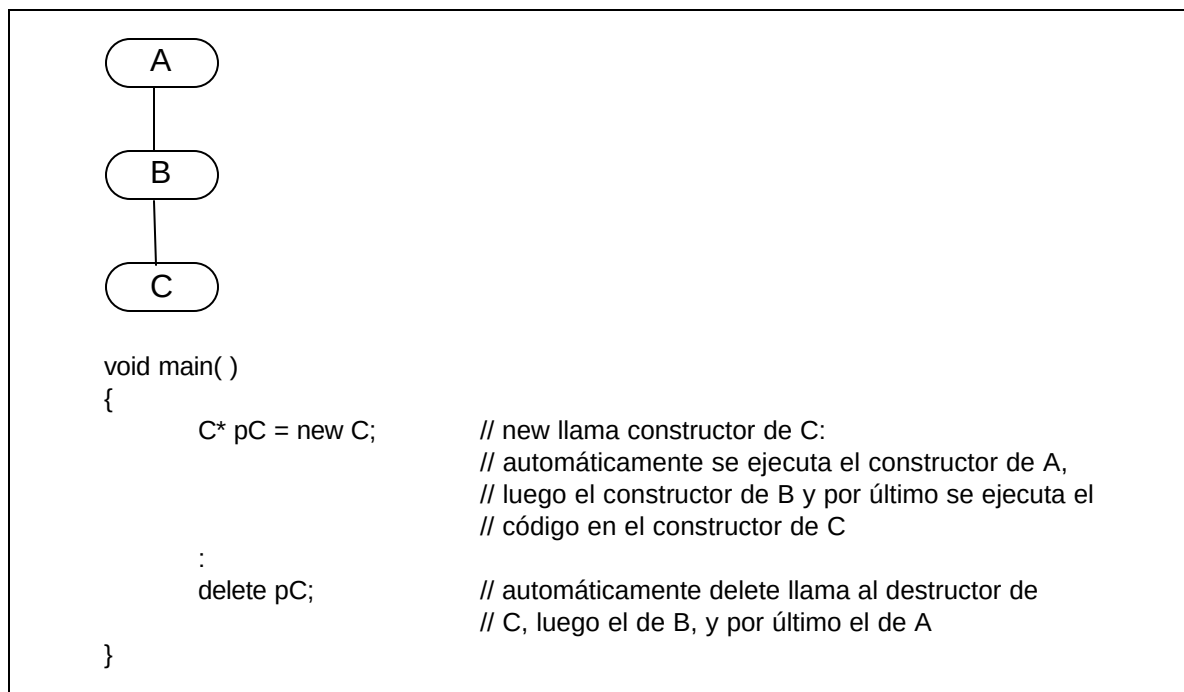
Una de las limitaciones de la herencia, tal cual es soportada hoy en día, está en que se debe definir en tiempo de diseño, y no puede ser dejada para tiempo de ejecución.

Lo que no puede ser heredado:

1. constructores
2. destructores
3. operadores (de asignación, comparación, etc.)
4. relación *friend*

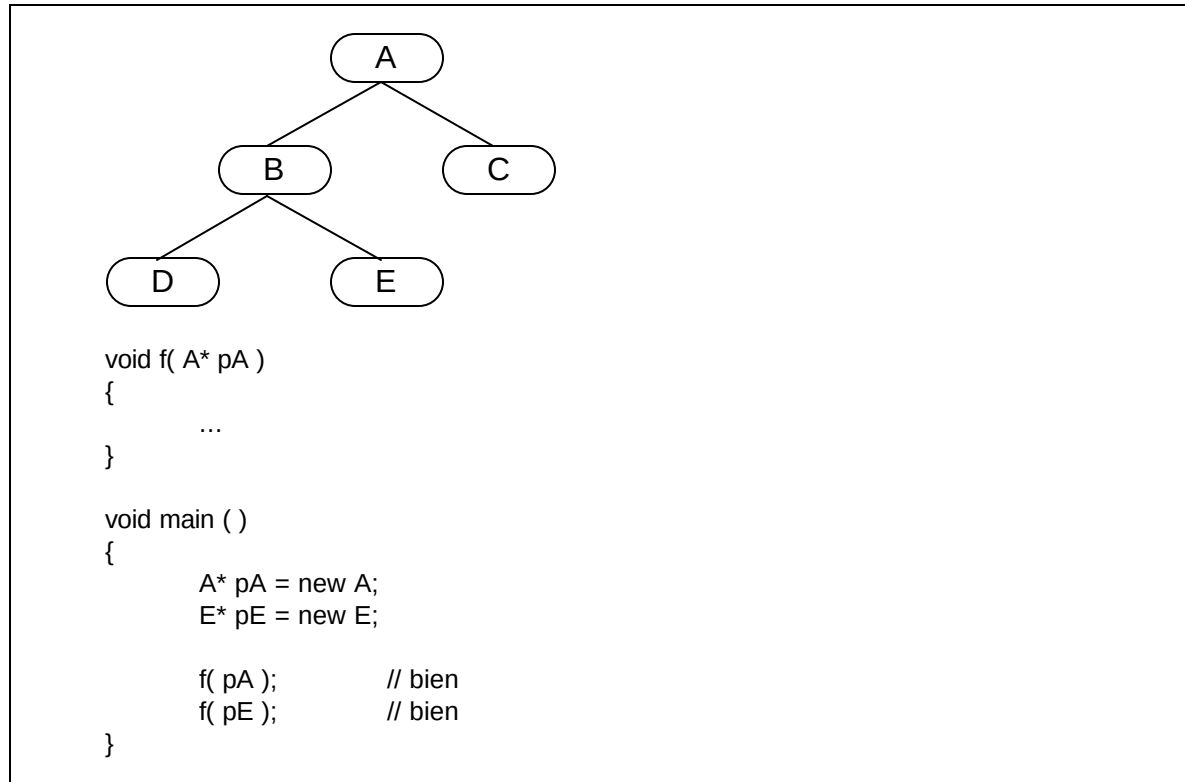
3.2 Constructores y destructores en el mecanismo de herencia

A continuación se muestra con un ejemplo la secuencia de ejecución de constructores y destructores en una jerarquía de herencia.

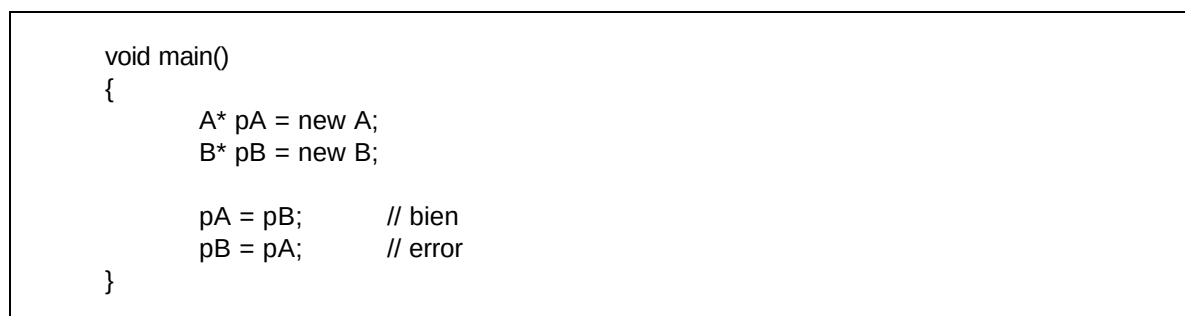


3.3 Conversiones implícitas de tipo

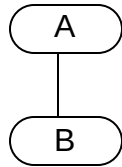
El estándar de C++ establece: "Siempre que se requiera una referencia a un objeto de una clase se puede utilizar una referencia a un objeto de esa clase, **o una referencia a un objeto de cualquiera de las clases descendientes**".



Siempre se puede construir un objeto de una clase ancestral a partir de un objeto de una clase descendiente, pero no lo contrario:



3.4 Resolución de prevalencia



```

class A
{
public:
    int f() { return 0; }
};

class B : public A
{
public:
    int f() { return 1; }
};

void main( )
{
    A a;
    B b;

    int i = a.f();    // i = 0
    int j = b.f();    // j = 1
}
  
```

¿Cómo decide el compilador que función f() se debe ejecutar?

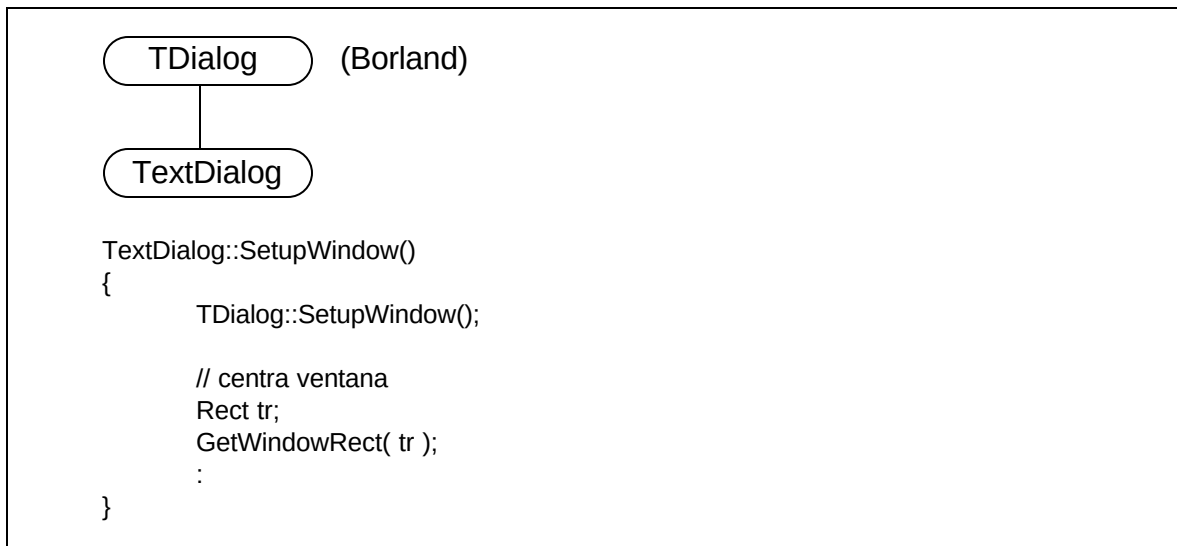
Busca la función f() que está más "cerca" de la clase B, comenzando en B y siguiendo hacia "arriba" en el árbol de herencia (si existe herencia múltiple la cosa se complica: más adelante). La función f() definida en la clase B redefine la función f() definida en la clase A. Si en la clase B se necesita ejecutar la función f() definida en A se utiliza el operador :: (scope resolutor):

```

void main( )
{
    A a;
    B b;

    int i = a.f();          // i = 0
    int j = b.A::f();       // j = 0
}
  
```

// Caso típico:



3.5 Herencia múltiple

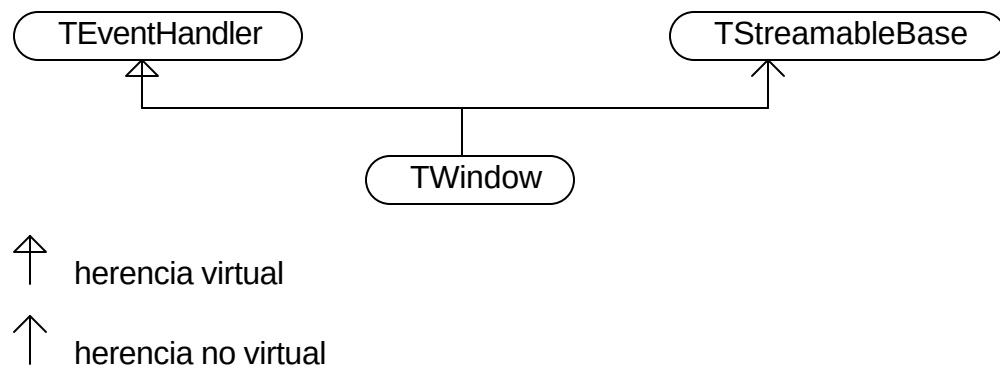
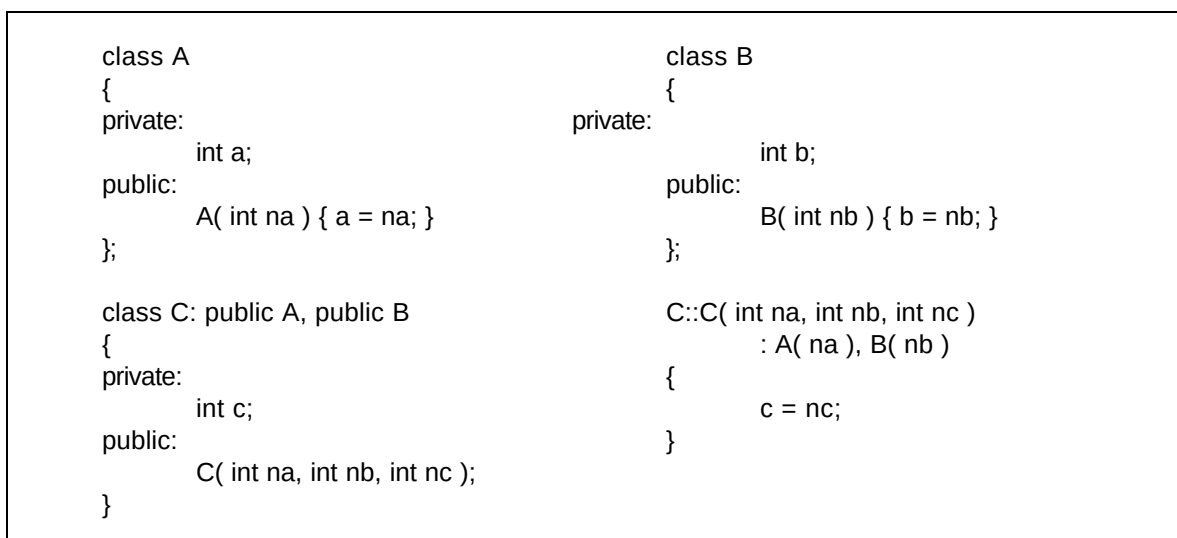


Fig. 4. Ejemplo de herencia múltiple en OWL de Borland



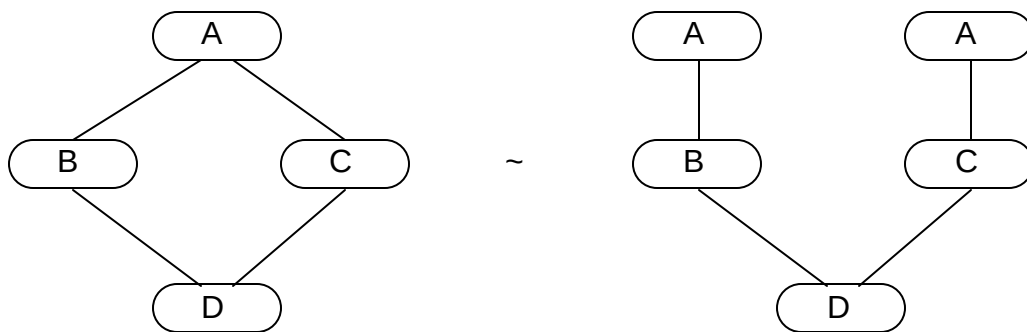
El orden en que se invocan los constructores depende del orden en el que se declara la clase C:

```
class C : public A, public B
{
    :
}
```

y no importa el orden de llamada de los constructores de A y B en el constructor de C.

3.6 Herencia virtual

La herencia virtual sólo se utiliza en el contexto de herencia múltiple. En el caso que se muestra a continuación la clase D es descendiente de la clase A. El problema está en que hereda dos clases A y por lo tanto dos enteros 'a'. Cuando en un método de D se referencia 'a', el compilador no puede resolver de cual de los dos 'a' heredados se trata. La forma de solucionar estas situaciones es por medio de la herencia virtual, como se muestra a continuación:



```
class A
{
protected:
    int a;
};

class B : public A {};
class C : public A {};

class D : public B, public C
{
public:
    int f() { return a; }    // error:
                          // "Field 'a' is ambiguous in class 'D' in function D::f()"
};

class D : public B, public C
{
public:
```

```
        int f() { return C::a; }    // bien  
};
```

// o mejor:

```
class D: public virtual B, public virtual C  
{  
public:  
    int f() { return a; }        // bien  
};
```

// o también:

```
class A  
{  
protected:  
    int a;  
};  
  
class B: virtual public A {};  
class C: virtual public A {};  
class D: public B, public C  
{  
public:  
    int f() { return a; }        // bien  
};
```

4. Sobrecarga

La sobrecarga es un mecanismo que permite utilizar un mismo nombre de función para construir diferentes funciones: diferentes argumentos y diferentes comportamientos. Es utilizado comúnmente para definir diferentes constructores de una clase, esto es, para construir objetos de una misma clase pero de formas diferentes. Como veremos, también se utiliza en los métodos consultores y modificadores de un atributo de una clase.

4.1 Sobrecarga de funciones

La sobrecarga de funciones básicamente se refiere a la posibilidad de asignarle más de un comportamiento a una función de una clase. Cuanto menos detalles sean necesarios para utilizar un sistema, una función en este caso, mejor.

```
void print( char c )      { printf( "%c", c ); }      // el usuario de printf() debe conocer
void print( int i )      { printf( "%d", i ); }      // y recordar todos los formatos
void print( char* sz )   { printf( "%s", sz ); }
void print( float f )    { printf( "%f", f ); }
void print( double d )   { printf( "%Lf ", d ); }

void main ( )
{
    print( 3.14159 ); // bien          // el usuario de print() no
    print( "número" ); // bien
    print( 'c' );      // bien
}
```

Funciones con igual nombre son diferenciadas por el compilador (i.e. en tiempo de compilación), según la cantidad y tipo de argumentos, no por el valor de retorno:

```
void print( char c );
char print( char c );      // error
void print( char c, int i ); // bien
```

En clases:

```
class Pixel : public Punto
{
private:
    int color;
```

```

public:
    Pixel( int nx, int ny, int ncolor );

    void Color( int ncolor ) { color = ncolor; } // cambia el color
    int Color( ) { return color; }              // retorna el color
    :
};

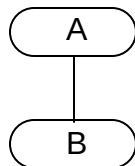
void main()
{
    Pixel p( 0, 0, 0 );

    :

    if ( p.Color() == 0 )
    {
        p.Color( 5 );
    }
}

```

4.2 Sobrecarga ¹ Redefinición



```

class A
{
public:
    void f( int i ) { printf( "%d", i ); }
};

class B : public A
{
public:
    void f( float f ) { printf( "%f", f+10 ); }
};

void main ( )
{
    B b;
    b.f( 1 );          // pinta 11
}

```

```
// la función f( float f ) en B redefine,
// no sobrecarga, la función A::f( int i )
// overriding vs. overloading
```

4.3 Sobrecarga de operadores

Los operadores (+, -, *, /, =,), están definidos en el lenguaje C++ para 'operar' sobre tipos intrínsecos (char, int, float, etc.), pero pueden ser sobrecargados para operar sobre otros objetos definidos por el usuario.

Notación simple y uniforme:

```
string s1 = "Universidad";
string s2 = "Nacional";
string s3 = s1 + s2;           // como en Pascal

// c estándar:

strcpy( s3, s1 );
strcat( s3, s2 );

// o también:

sprintf( s3, "%s %s", s1, s2 );
```

Los operadores pueden ser implementados como funciones miembro o como funciones *friend*. Por ejemplo, para un objeto 'x' y el operador unario '!', la expresión

!x es equivalente a

```
x.operator!()           // como función miembro
operator!( x )          // como función friend
```

Para los objetos 'u', 'v' y el operador binario '+', la expresión

u + v equivale a la llamada de función

```
u.operator+( v )         // como función miembro
operator+( u, v )        // como función friend
```

En todo caso, el nombre comienza siempre con '*operator*', y después el operador que se desea sobrecargar.

4.3.1 Sobrecarga de operadores por medio de funciones miembro

1. si es un operador unario no puede recibir parámetros.
2. si es un operador binario sólo puede recibir un parámetro.
3. el valor de retorno puede ser cualquiera, sin embargo se estila hacer que retornen una referencia a un objeto de la misma clase, con el fin de que el compilador pueda evaluar expresiones tales como:

$$w = x + y + z;$$

```
class Ejemplo
{
private:
    int a;
public:
    Ejemplo( int na ) { a = na; }
    Ejemplo& operator!();
    Ejemplo& operator+( Ejemplo& e );
};

Ejemplo&
Ejemplo::operator!()
{
    a = !a;
    return *this;
}

Ejemplo&
Ejemplo::operator+( Ejemplo& e )
{
    a += e.a;
    return *this;
}

void main ( )
{
    Ejemplo e( 0 )
    Ejemplo f( 5 );

    e = !e;          // e.a ← -32768
    e = e + f;       // e.a ← -32763
}
```

4. sí pueden ser virtuales (más adelante)
5. no pueden ser *static* (no reciben apuntador *this*)

4.3.2 Sobrecarga de operadores por medio de funciones *friend*

```
class Ejemplo
{
    friend Ejemplo& operator+( Ejemplo& e, Ejemplo& f );
private :
    int a;
public:
    Ejemplo( int na ) { a = na; }
    Ejemplo& operator=( Ejemplo& e );    // el operador = no puede ser friend
};

Ejemplo&
Ejemplo::operator=( Ejemplo& e )
{
    a = e.a;
    return *this;
}

Ejemplo&
operator+( Ejemplo& e, Ejemplo& f )
{
    e.a += f.a;
    return e;
}
```

El operador de asignación (=) no puede ser implementado con una función *friend* de la clase, únicamente como función miembro. Esto implica que el operador global '=' no puede ser sobrecargado.

```
int a, b, c;
a = b + c;    // operator=( int, int ) utilizado

float f, g, h;
f = g + h ;    // operator=( float, float ) utilizado
```

También hay que tener en cuenta que los operadores no son conmutativos.

4.4 Constructor de gemelos (copy constructor)

Este tipo de constructor es utilizado para crear objetos como copias idénticas (o casi) de otro.

```
class Ejemplo
{
private :
    int a;
    :
public:
    Ejemplo( int na ) { a = na; }
    Ejemplo( Ejemplo& e );           // copy constructor
    Ejemplo& operator=( Ejemplo& e ); // operador de asignación
};

Ejemplo::Ejemplo( Ejemplo& e )
{
    a = e.a;
    :
}

void main ( )
{
    Ejemplo e( 1 );
    Ejemplo f( 5 );

    e = f;           // operator=( Ejemplo& ) utilizado
    Ejemplo g = e;   // copy constructor utilizado
                    // en este caso se declara y se define un objeto de la
                    // clase Ejemplo
}

// o también:

Ejemplo::Ejemplo( Ejemplo& e )
{
    *this = e;       // copy constructor que usa el operador=( Ejemplo& )
}
```

Si no se define un operador de asignación (=) el compilador genera uno automáticamente (si es necesario, sino no), que efectúa una copia uno a uno de todos los componentes de la clase.

Nota: el operador de asignación definido en una clase no es heredado por sus clases descendientes.

Por interferir con el lenguaje no se pueden sobrecargar los siguientes operadores:

- . operador punto (referencia de componentes)
- .* operador dereferenciador (dereferencia a un apuntador miembro)
- :: operador de resolución
- ? : operador ternario condicional ($(a < b) ? a : b$)

Los operadores *new* y *delete* pueden ser sobrecargados.

Más adelante (sección 7, *streams*) veremos que la sobrecarga de operadores es de gran utilidad en el manejo de entrada/salida, donde la funcionalidad está dada por medio de la sobrecarga de los operadores de inserción/extracción "<<" y ">>" respectivamente. De esta manera se logra un manejo uniforme de todas las operaciones de escritura/lectura de objetos.

5. Polimorfismo

Junto con el mecanismo de herencia, el polimorfismo es la otra herramienta en que se basa el poder de la programación orientada por objetos:

Poly → muchos
Morphos → formas

Se refiere en este caso a la capacidad que tienen las funciones miembro de tomar diferentes formas en diferentes clases. La forma de declarar una función polimórfica es antecediéndole el calificativo *virtual*.

```
class ObjetoGrafico
{
protected:
    TRect bbox;
public:
    ObjetoGrafico( TRect tr );
    ObjetoGrafico( ObjetoGrafico& og );
    :
    virtual void Display( TDC tdc ) = 0;    // notación nueva, más adelante
};

class Rectangulo : public ObjetoGrafico
{
public:
    Rectangulo( TRect tr ) : ObjetoGrafico( tr ) {}
    :
    void Display( TDC& tdc );
}

class Elipse: public ObjetoGrafico
{
public:
    Elipse( TRect tr ) : ObjetoGrafico( tr ) {}
    :
    void Display( TDC& tdc );
}

class Texto: public ObjetoGrafico
{
private:
    char* szText;
public:
    Texto( TRect tr, char* sz );
    :
    void Display( TDC& tdc );
}
```

```

    }
    void
    Rectangulo::Display( TDC& tdc )
    {
        :
        tdc.Rectangle( bbox );
    }

    void
    Elipse::Display( TDC& tdc )
    {
        :
        tdc.Ellipse( bbox );
    }

    void
    Texto::Display( TDC& tdc )
    {
        tdc.SelectFont( ... );
        :
        tdc.TextOut( szText, ... );
    }

```

Entonces, si los objetos gráficos creados por el usuario son almacenados en una lista, la forma de pintar el "dibujo" será recorrer la lista y enviarle un mensaje a cada objeto gráfico: píntese (Display()).

```

void
Dibujo::Display( TDC& tdc )
{
    ListIterator it( listaObjetos );

    while ( it )
    {
        it.Current.Display( tdc );      // nada de switch/case ...
        it++ ;
    }
}

```

Ahora, ¿cómo sabe el compilador, de que clase es el objeto referido por el iterador (it) en tiempo de ejecución?. El usuario puede haber agregado al dibujo una cantidad arbitraria (0, 1, ... n) de objetos de cada clase. No se puede saber en tiempo de compilación/encadenamiento: será

resuelto en tiempo de ejecución. Esto está indicado por el calificativo *virtual* al comienzo de la función y el mecanismo se conoce como *late binding* o encadenamiento tardío.

La declaración en la clase ObjetoGrafico:

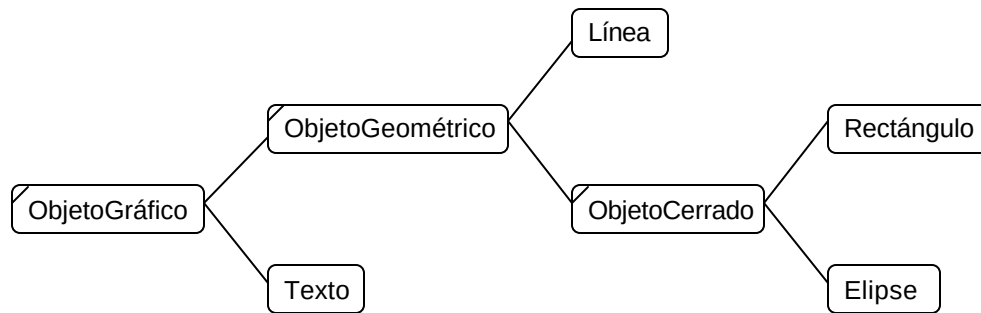
```
:
virtual void Display( TDC tdc ) = 0;      // método abstracto
```

no solo especifica que el método Display va a ser redefinido en las clases descendientes (virtual) sino que obliga (= 0) a toda clase descendiente a definirlo. Un método así declarado en un método abstracto y no puede ser definido (¿cómo se pinta un ObjetoGrafico además?). Una clase que tiene uno o más métodos abstractos es una clase abstracta y tiene como objetivo encapsular el conocimiento y comportamiento común a las clases que de ella descienden.

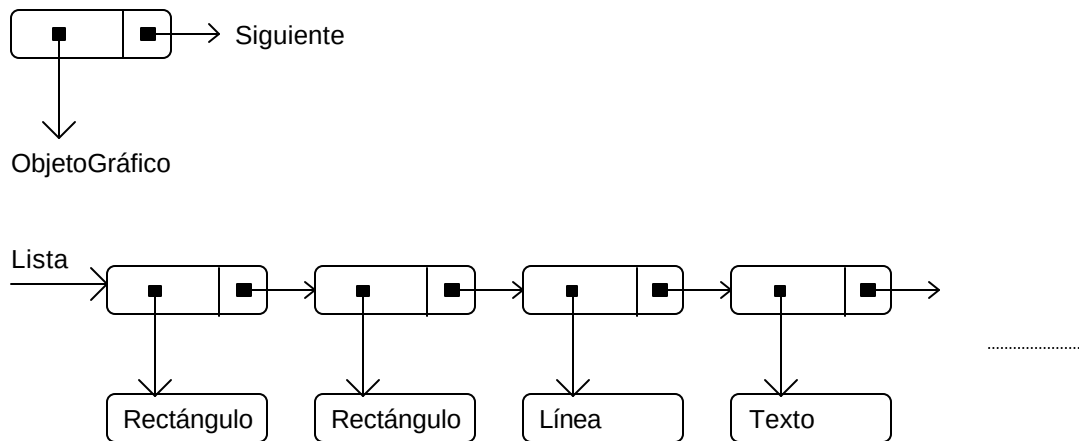
De una clase abstracta no se han de instanciar objetos y por lo tanto el compilador no lo permite. Por oposición, las clases construidas para generar objetos son llamadas 'clases concretas' y deben definir los métodos abstractos de sus superclases. La decisión de hacer uno de estos métodos virtual en una clase concreta depende de si se van a definir nuevas clases concretas descendientes de ésta. Si por ejemplo, se fuera a definir la clase Cuadrado como una clase especial de Rectángulo, entonces el método Display debería ser declarado virtual en esta última.

```
class Rectangulo : public ObjetoGrafico
{
    :
    virtual void Display( TDC& tdc ); // concreto pero redefinible
};
```

Los métodos constructores no pueden ser virtuales. Los métodos destructores, por el contrario, pueden ser virtuales y muchas veces es necesario que así sea. Un ejemplo se puede encontrar en la jerarquía de ObjetosGráficos.



En este caso los objetos gráficos que el usuario va creando son almacenados en una lista construida para referenciar ObjetosGráficos, aprovechando la compatibilidad que brinda el mecanismo de herencia, que nos permite almacenar después cualquier objeto de una clase descendiente de ObjetoGráfico:



Cuando se va a destruir la lista de objetos gráficos, ya sea porque el usuario comienza un nuevo dibujo o porque ha terminado la aplicación, ésta debe ser recorrida y cada elemento debe ser destruido:

```

void
Dibujo::flush()
{
    ListIterator it( Lista );

    while ( it )
    {

```

```

        delete it.Current();    // el destructor de ObjetoGráfico es llamado
        it++;                  // a menos que sea declarado virtual
    }
}

```

Como la lista ha sido construida para almacenar ObjetosGráficos, si el destructor de ObjetoGráfico no es declarado virtual, en el ejemplo de código anterior el operador delete llamará *siempre* al destructor de ObjetoGráfico. Como las clases Rectángulo, Línea y Texto agregan componentes a los declarados en ObjetoGráfico, los objetos de la lista son destruidos incorrectamente. Por lo tanto en la declaración de ObjetoGráfico se ha hecho lo siguiente:

```

class ObjetoGrafico
{
    :
    virtual ~ObjetoGrafico();
};

```

Otra vez, la decisión de hacer uno de estos métodos virtual en una clase concreta tal como Rectángulo, Línea o Texto, depende de si se van a definir nuevas clases concretas descendientes de alguna de éstas.

6. Bibliotecas de clases en Borland C++ 4.0

Borland C++ provee tres grandes bibliotecas de clases: la jerarquía de clases contenedoras (*Containers Class Library*), la jerarquía de *streams* y la jerarquía OWL (*ObjectWindows Library*). La jerarquía de clases contenedoras incluye las Estructuras Fundamentales (*Fundamental Data Structures*) y los Tipos Abstractos (*Abstract Data Types*). Esta jerarquía será analizada a continuación. Por su importancia, las jerarquía de *streams* y OWL serán estudiadas en las secciones 7 y 9 respectivamente.

6.1. Clases genéricas (*templates*)

Los *templates* son tipos genéricos, también llamados tipos parametrizados, y permiten la construcción de patrones para familias de funciones o clases relacionadas.

Consideremos por ejemplo el caso de la función $\text{max}(a, b)$, que retorna el mayor de sus argumentos a y b , donde éstos son de cualquier tipo ordenable. En lenguaje C para poder ordenar los diferentes tipos es necesario definir diferentes funciones:

```
int
max_int( int a, int b )
{
    return (a > b) ? a : b;
}

long
max_long( long a, long b )
{
    return (a > b) ? a : b;
}

etc.
```

En C++ se podría aprovechar el mecanismo de sobrecarga de funciones y en este caso no sería necesario buscar diferentes nombres. Sin embargo con este enfoque también se debe definir una función para cada tipo:

```
int
max( int a, int b )
{
    return (a > b) ? a : b;
}
```

```

long
max( long a, long b )
{
    return (a > b) ? a : b;
}

etc.

```

Lo anterior se puede evitar con el clásico uso de una macro,

```

#define max( a, b )    ( ((a) > (b)) ? (a) : (b) )

```

pero esto conlleva el problema de que no se está aprovechando el mecanismo de verificación de tipos que ofrece el C++, y se podrían estar comparando dos tipos incompatibles. Para solucionar este tipo de situaciones C++ permite definir funciones genéricas siguiendo la sintaxis que se muestra a continuación:

```

template
<class T> T
max( T a, T b )
{
    return (a > b) ? a : b;
}

```

De esta manera se puede utilizar la función *max(a, b)* con cualquier clase de objetos que haya sobrecargado apropiadamente el operador >:

```

class ClaseX
{
    :
    ClaseX& operator>( const ClaseX& b );
};

ClaseX&
ClaseX::operator>( const ClaseX& b )

```

```

{
    if ( ... )
    {
        return *this;
    }
    else
    {
        return b;
    }
}

int i;
ClaseX a, b;

int j = max( i, 0 );
ClaseX c = max( a, b );

```

Así como se pueden definir funciones genéricas, en C++ se pueden construir clases genéricas. Un buen ejemplo de uso de estas clases se encuentra en la jerarquía de clases contenedoras que provee Borland C++.

6.2. Clases contenedoras (*containers*)

Las estructuras de datos provistas por Borland (*BIDS, Borland International Data Structures*) están agrupadas en la biblioteca de clases contenedoras. La sintaxis para la declaración de una clase parametrizada se muestra a continuación con la clase *Vector*, implantada como una matriz uni-dimensional:

```

template <class T> class Vector
{
public:
    Vector( int );
    ~Vector() { delete [] data; }
    T& operator[]( int i ) { return data[ i ]; }
private:
    T*    data;
    int    size;
};

template <class T> Vector<T>::Vector( int i )
{
    size = i;
    data = new T[ size ];
}

```

```
// ejemplo de uso

void main()
{
    Vector<int> i( 5 );
    Vector<ObjetoGrafico> o( 100 );
    :
}
```

Las clases contenedoras, como su nombre lo indica, son utilizadas para almacenar objetos en memoria. La estructura subyacente utilizada por un contenedor determina la forma en que los objetos son almacenados y accedidos.

Los contenedores pueden almacenar objetos de dos maneras: directa o indirectamente. En el modo de almacenamiento directo el objeto es efectivamente copiado en el contenedor. En el modo de almacenamiento indirecto se utiliza un apuntador que referencia al objeto almacenado. El modo de almacenamiento a utilizar dependerá del tamaño de los objetos a almacenar: objetos pequeños en forma directa y objetos grandes en forma indirecta.

Borland C++ provee una biblioteca de clases parametrizadas que encapsula los detalles de almacenamiento físico. De esta manera se puede optimizar el uso de memoria en una aplicación, con un mínimo de reprogramación. Estas clases están construidas a partir de Estructuras Fundamentales (*Vector*, *List*, *BTree*,...) y utilizan Tipos Abstractos (*Array*, *Stack*, *Queue*,...) como estructuras subyacentes. La nomenclatura utilizada por la clases contenedoras es la siguiente:

```
template <class [T, I, V, Alloc, List, Stk, Vect]> class _EXPCLASS ContainerClassName ...
```

donde:

T	significa	Template
I		Indirect (apuntadores)
V		Virtual
Alloc		Allocator
List		List
Stk		Stack
Vect		Vector

Las Estructuras Fundamentales son contenedores de bajo nivel que pueden ser instanciados o utilizados para construir Tipos Abstractos (ADTs). Las clases de este tipo provistas por Borland son las siguientes:

BTree	árboles binarios no balanceados, útiles en el almacenamiento de objetos de gran tamaño (binimp.h)
DoubleList	los objetos son almacenados en una lista lineal que puede ser recorrida en ambas direcciones (dlistimp.h)
HashTable	una colección no ordenada donde el almacenamiento y acceso de los objetos está determinado por un valor único (<i>hash value</i>) (hashimp.h)
List	los objetos son almacenados en una lista lineal que puede ser recorrida en una sola dirección (listimp.h)
Vector	almacenamiento indizado, el índice utilizado comienza en 0 y son útiles cuando se conoce de antemano la cantidad de objetos a almacenar (vectimp.h)

Los Tipos Abstractos (ADTs) son contenedores de alto nivel que tienen por lo menos un método virtual. De esta forma, los Tipos Abstractos no pueden ser instanciados sin una Estructura Fundamental subyacente. Las clases de este tipo provistas por Borland son las siguientes:

Array	almacenamiento indizado, el índice utilizado comienza en 1 y no es necesario conocer de antemano la cantidad de objetos a almacenar (array.h)
	ejemplos: TArrayAsVector TArrayAsVector TISArrayAsVector ..
Bag	los objetos no son almacenados según un orden particular, no es necesario que sean ordenables y se pueden tener múltiples instancias (bags.h)
	ejemplos: TBagAsVector TIBagAsVector TMBagAsVector ..
Deque	doble cola, los objetos son almacenados o accedidos ya sea por la cabeza o por la cola del contenedor (deques.h)
	ejemplos: TDequeAsDoubleList TDequeAsVector ..
Dictionary	estructura que almacena parejas de objetos relacionados (dict.h)
	ejemplos: TDictionary TDictionaryAsHashTable

	TIDictionaryAsHashTable ..
Queue	cola, los objetos son almacenados por la cola y accesados por la cabeza del contenedor (queues.h)
	ejemplos: TQueue TQueueAsVector TQueueAsDoubleList ..
Set	los objetos no son almacenados según un orden particular, no es necesario que sean ordenables y sólo se puede tener una instancia (sets.h)
	ejemplos: TSet TSetAsVector TISetAsVector ..
Stack	pila, los objetos son almacenados y accesados por la cabeza del contenedor (stacks.h)
	ejemplos: TStack TStackAsVector TStackAsList ..

En el graficador implementado en este curso aparecen ejemplos de uso de Tipos Abstractos utilizados en el almacenamiento de Eventos (stacks) y de ObjetosGráficos (sets). El uso recomendado está en definir un nuevo tipo con el fin de poder cambiar la implementación específica de los contenedores utilizados con sólo modificar una línea de código:

```
// VEdicion.hpp
class VentanaEdicion : publicTWindow
{
    :
private:
    typedef TISetAsVector<ObjetoGrafico> Set;

    Stack* undoStack;
    Stack* redoStack;
};

// Dibujo.hpp
class Dibujo
{
    :
private:
    typedef TISetAsVector<ObjetoGrafico> Set;
```

```

        typedef TSetAsVectorIterator<ObjetoGrafico> SetIterator;
        Set*   dibujo;
        Set*   grupo;
        Set*   clipboard;
};

```

El acceso a los elementos almacenados en un contenedor se hace por medios de *iteradores*. Cada clase contenedora tiene asociado un iterador, por ejemplo, la clase TArrayAsVector tiene el iterador correspondiente TArrayAsVectorIterator, responsable de permitir *iterar* sobre los objetos en el contenedor.

Todos los iteradores implementan los operadores ++ de pre y post incremento. También proveen la función *Current()* que retorna el objeto referenciado por el iterador en cada instante y la función *Restart()* que permite reinicializar el iterador. En el graficador de ejemplo también aparece el uso de iteradores para acceder los ObjetosGráficos seleccionados:

```

// Dibujo.cpp

void
Dibujo::showGroupBBox( TDC& tdc, TPoint& delta )
{
    SetIterator it( *grupo );

    while ( it )
    {
        it.Current()->showBBox( tdc, delta );
        it++;
    }

} // Dibujo::showGroupBBox

void
Dibujo::copiar()
{
    SetIterator it( *grupo );

    // clipboard de Windows
    while ( it )
    {
        :
        it++;
    }

    it.Restart();
}

```

```
        // clipboard privado
        while ( it )
        {
            :
            it++;
        }

    } // Dibujo::copiar
```

```
Dibujo::Dibujo( VentanaEdicion* pOwner )
{
    :

    dibujo = new Set;
    dibujo->OwnsElements( TRUE );

    grupo = new Set;
    grupo->OwnsElements( FALSE );

    clipboard = new Set;
    clipboard->OwnsElements( TRUE );

} // Dibujo::constructor
```

7. Streams

El lenguaje C++ presenta una nueva técnica para el manejo de entrada/salida: *streams*. Los *streams* de C++ proveen la funcionalidad de la biblioteca *stdio* (*standard input/output*) del lenguaje C, más un conjunto de nuevas facilidades. En su época el lenguaje C marcó la diferencia al modelar la entrada/salida por medio de archivos. A pesar de que en C++ los *streams* están asociados comúnmente al manejo de entrada/salida, un *stream* es una abstracción que refiere cualquier flujo de datos entre un objeto 'productor' y un objeto 'consumidor'. Ahora las operaciones de entrada/salida son llevadas a cabo en un ámbito orientado por objetos y la funcionalidad es provista por medio de la sobrecarga de los operadores de inserción/extracción << y >>. Si bien es posible utilizar *streams* y ANSI I/O en un mismo programa, esto no es recomendable ya que pueden aparecer problemas de sincronismo en el manejo de buffers.

La biblioteca *iostream* está compuesta por dos familias paralelas de clases: las clases derivadas de *streambuf* y las derivadas de *ios* (ver figura 7.1).

La clase *streambuf* provee la interfaz necesaria con memoria y con dispositivos físicos. Los métodos de las clases de la familia *streambuf* son utilizados por las clases de la familia *ios*, a través de un apuntador. Todas las clases basadas en *ios* utilizan un objeto *streambuf* (o *filebuf* o *strstreambuf*, que son casos especiales) como fuente o destino.

Todo programa construido en lenguaje C cuenta, al iniciar su ejecución, con tres archivos abiertos: *stdout*, para salida en pantalla, *stdin*, para lectura de teclado, y *stderr*, para despliegue de errores. De forma similar, todo programa en C++ inicia su ejecución con los siguientes *streams* creados:

```
extern istream_withassign cin;    // corresponde a stdin
extern ostream_withassign cout;  // corresponde a stdout
extern istream_withassign cerr;  // corresponde a stderr
extern istream_withassign clog;  // stderr con soporte de buffer
```

Borland C++ adhiere el estándar de AT&T 2.1 y provee las siguientes clases para el manejo de *streams*:

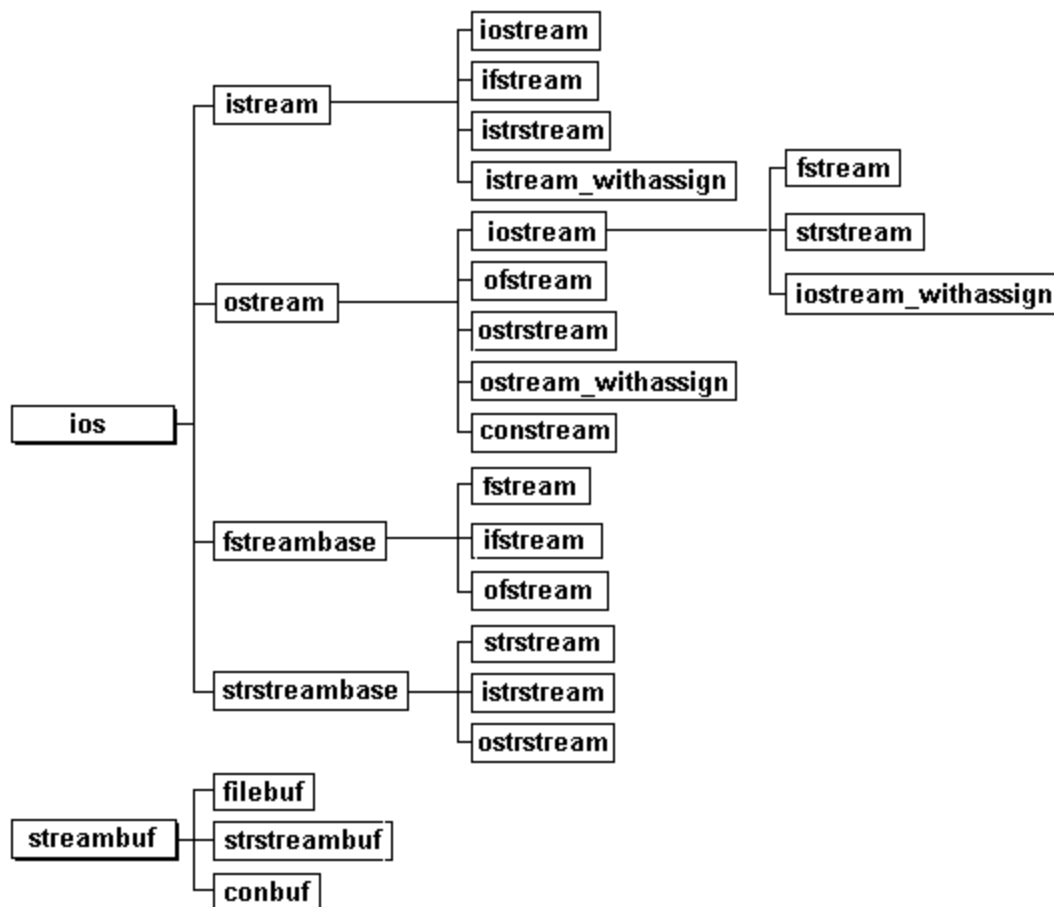


Fig. 7.1. Jerarquía de clases *streams*.

Las operaciones globalmente conocidas como de entrada/salida pueden ser agrupadas en tres grandes categorías:

Categoría	Ejemplo de función en C
E/S estándar	<code>scanf(...)</code> , <code>printf(...)</code> , ...
E/S de archivos	<code>fopen(...)</code> , <code>fwrite(...)</code> , <code>fread(...)</code> , <code>fclose(...)</code> , ...
Formato en memoria	<code>sscanf(...)</code> , <code>sprintf(...)</code> , ...

A continuación se presentan las operaciones más comunes en cada una de estas categorías, utilizando *streams* y utilizando la biblioteca de funciones *stdio*.

7.1. Entrada/Salida estándar

```
void main()
{
    // C++
    int i;
    float f;
    cout << "valores? ";
    cin >> i;
    cin >> f;
    cout << "valores: " << i << ", " << f;
}

// C
// printf( "valores? " );
// scanf( "%d", &i );
// scanf( "%f", &f );
// printf( "valores: %d, %f", i, f );
```

En el ejemplo anterior se puede apreciar la uniformidad en la notación utilizada tanto para operaciones de entrada como para operaciones de salida. Ya no se corre el riesgo de cometer los siguientes errores (que no son detectados por el compilador):

```
int i;
double d;

scanf( "%d", i );           // parámetro pasado por valor!
printf( "%lf", d ); // cadena de formato errónea
```

Las secuencias de escape (*escape sequences*) utilizadas en C también pueden ser utilizadas en *streams*:

\a	sonido (<i>beep</i>)
\b	borra caracter anterior (<i>backspace</i>)
\f	avanza página (<i>form feed</i>)
\n	cambio de línea (<i>new line character</i>)
\r	enter (<i>carriage return</i>)
\t	tabulado horizontal (<i>horizontal tab</i>)
\v	tabulado vertical (<i>vertical tab</i>)
\\	oblicua (<i>backslash character</i>)
\'	comilla simple (<i>single quotation mark</i>)
\"	comilla doble (<i>double quotation mark</i>)

\O	base octal
\x	base hexadecimal (minúsculas)
\X	base hexadecimal (mayúsculas)
\0	terminador nulo

Con el fin de mantener la uniformidad en la notación, el manejo de entrada/salida estándar puede ser fácilmente extendido para manipular clases, aprovechando la sobrecarga de operadores que provee el lenguaje C++:

```
#include <iostream.h>

class Punto3D
{
public:
    Punto3D( double dx, double dy, double dz ) { x = dx; y = dy; z = dz; }
    :
    friend ostream& operator<<( ostream& os, Punto3D& p );
    friend istream& operator>>( istream& is, Punto3D& p );

private:
    double x;
    double y;
    double z;
};

ostream& operator<<( ostream& os, Punto3D& p )
{
    return os << "( " << p.x << ", " << p.y << ", " << p.z << " )";
}

istream& operator>>( istream& is, Punto3D& p )
{
    return is >> p.x >> p.y >> p.z;
}

void main()
{
    Punto3D p( 1, 2, 3 );

    cout << "coordenadas de p = " << p;    // coordenadas de p = ( 1, 2, 3 )
    cout << "nuevas coordenadas? ";
    cin >> p;                               // lee nuevas coordenadas
}
```

7.2. Manipuladores

En lenguaje C se puede dar formato a la información (que se va a enviar a pantalla o a un archivo) por medio de las cadenas de formato utilizadas en funciones como *printf()* o *sprintf()*.

```
#include <stdio.h>
:
int i = 15;
printf( "i = %X", i );           // i = 0x0F
```

En C++ el formato a la información se da por medio de *manipuladores*. Para hacer uso de los manipuladores se debe incluir el archivo de prototipos *iomanip.h*.

```
#include <iostream.h>
#include <iomanip.h>
:
int i = 15;
cout << "i = " << hex << i;      // i = 0x0F
```

A continuación se presenta una lista de los manipuladores disponibles.

dec	Fija la base de conversión decimal. Esta es la base activa por omisión.
hex	Fija la base de conversión hexadecimal.
oct	Fija la base de conversión octal.
ws	Extrae los caracteres en blanco de un <i>stream</i> de entrada.
endl	Inserta una nueva línea en un <i>stream</i> de salida.
ends	Inserta '\0' en un <i>stream</i> de salida.
setbase(int)	Fija la base de conversión: 0 - decimal 8 - octal 10 - decimal 16 - hexadecimal
resetioflags(long f)	Borra los bits del formato especificado por f.

<code>setioflags(long f)</code>	Fija los bits del formato especificado por <code>f</code>, que puede tomar los siguientes valores: <pre> ios::skipws ios::left ios::right ios::internal ios::dec ios::oct ios::hex ios::showbase ios::showpoint ios::uppercase ios::showpos ios::scientific ios::fixed ios::unitbuf ios::stdio </pre>
<code>setfill(int)</code>	Fija el caracter de relleno. Espacio en blanco por omisión.
<code>setprecision(int)</code>	Establece la cantidad de cifras decimales a utilizar. Sólo para <i>streams</i> de salida.
<code>setw(int width)</code>	Fija el ancho (en caracteres) de la próxima variable a ser insertada en un <i>stream</i> de salida. Si la variable ocupa menos espacio se completa el campo según <code>setfill(int)</code> .

Si se necesita cierta funcionalidad no provista por los manipuladores estándar, se pueden crear nuevos manipuladores.

```

// Ejemplo

#include <iostream.h>

ostream& pesos( ostream& os )
{
    return os << "\t$ ";
}

void main()
{
    float f = 123.45;
    cout << pesos << f;           //      $ 123.45
}

```

7.3. Manejo de archivos

La simplicidad provista por la biblioteca de *streams* para el manejo de entrada/salida estándar también está disponible para el manejo de archivos, ya sea de tipo texto o binarios. Esta funcionalidad está soportada principalmente por dos clases: *ifstream* (entrada) y *ofstream* (salida). Su utilización se presenta a través de los siguientes ejemplos.

```
// Ejemplo: copia de archivos ASCII

#include <fstream.h>

void main()
{
    ifstream ifs( "entrada.txt" );    // crea el objeto ifs y abre el archivo
    ofstream ofs( "salida.txt" );    // idem

    if ( !ifs ) cerr << "No se pudo abrir archivo de entrada";
    if ( !ofs ) cerr << "No se pudo crear archivo de salida";

    char c;
    while ( ofs && ifs.get( c ) )
    {
        ofs.put( c );
    }
}

// Otro ejemplo

void main()
{
    ifstream ifs;                    // crea el objeto

    ifs.open( "entrada.txt" );        // abre el archivo
    if ( !ifs )                       // equivalente a: if ( ifs.bad() ) ...
    {
        cerr << "No se pudo abrir archivo de entrada";
        return;
    }

    char buf[ 128 ];
    while ( ifs )
    {
        ifs.getline( buf, sizeof( buf ) );
        cout << buf << "\n";
    }

    ifs.close();
    ifs.open( "otro.txt" );           // ifs puede ser reutilizado
    :
```

```
}
```

```
// Ejemplo: archivos binarios

#include <fstream.h>

class Estudiante
{
public:
    Estudiante( char* nom, float prom );
    {
        strcpy( nombre, nom );
        promedio = prom;
    }

    char    nombre[ 80 ];
    float   promedio;
}

void main()
{
    ofstream ofs( "salida.bin", ios::binary ); // crea el objeto ofs y abre el archivo

    if ( !ofs ) cerr << "No se pudo crear archivo de salida";

    Estudiante e( "José Pérez", 3.0 );
    ofs.writeString( e.nombre );
    ofs << promedio;
}

// o mejor:

class Estudiante
{
public:
    Estudiante( char* nom, float prom );
    {
        strcpy( nombre, nom );
        promedio = prom;
    }

    friend ostream& operator<<( ostream& os, Estudiante& e );
    :
private:
    char    nombre[ 80 ];
    float   promedio;
}
```

```

ostream& operator<<( ostream& os, Estudiante& e )

{
    os.writeString( e.nombre );
    os << promedio;
}

void main()
{
    ofstream ofs( "salida.bin", ios::binary ); // crea el objeto ofs y abre el archivo

    if ( !ofs ) cerr << "No se pudo crear archivo de salida";

    Estudiante e( "José Pérez", 3.0 );
    ofs << e;
}

```

7.4. Formato de cadenas de caracteres en memoria

El formato de cadenas de caracteres en memoria se hace a través de la clase *stringstream* (***string stream***) para entrada/salida, y en casos particulares *istringstream* u *ostringstream*. El manejo de estas clases es similar al presentado para archivos.

```

// Ejemplo: sprintf(...) en C++

#include <strstream.h>

void main()
{
    stringstream str;           // va pidiendo memoria dinámica a medida
                                // que la va necesitando

    float f = 1.234;
    str << "valor = "
        << setprecision( 2 )
        << f
        << ends;                // terminador '\0'

    cout << str.rdbuf();        // valor = 1.23

    str.seekp( 0 );             // reinicializa punto de inserción
    str << ...                   // se reutiliza
}

```

8. Persistencia

Los objetos de interfaz de una aplicación, tales como ventanas, cajas de diálogo y demás, o los objetos que una aplicación manipula, como los ObjetosGráficos en el caso de nuestro ejemplo, son creados en tiempo de ejecución y destruidos al terminar el programa. Estos objetos pueden ser creados con la capacidad de ser transportables (*streamable*) y por lo tanto escritos en (transportados a) disco. De esta manera se dice que los objetos persisten a la aplicación, o de otra forma, que son *persistentes*.

Borland C++ 4.0 provee un mecanismo eficiente y consistente para crear clases persistentes. En primer lugar, la clase debe hacerse descendiente de *TStreamableBase*, y para esto se debe incluir `<objstrm.h>`. Esta clase es provista por Borland y se encarga de las operaciones básicas que debe llevar a cabo una clase para ser persistente. Borland también provee otras clases relacionadas con el manejo de persistencia, tales como *ObjectBuilder*, *TStreamableClass*, *TStreamableTypes*, *TStreamer* y otras, pero no es necesario trabajar directamente con ellas.

Toda clase persistente debe incluir un método para escribirse en disco (*write()*), un método para leerse de disco (*read()*) y un constructor con parámetro *StreamableInit* (*buid constructor*). Para simplificar la declaración de estos métodos Borland provee diferentes macros basadas en `DECLARE_STREAMABLE` y `IMPLEMENT_STREAMABLE`. Su uso dependerá de si la clase que se desea hacer persistente es una clase abstracta o no, y de otros parámetros que se muestran a continuación, utilizando como ejemplo la jerarquía de ObjetosGráficos del programa ejemplo. La macro `DECLARE_STREAMABLE` se expande a su vez en las siguientes macros:

```
#define DECLARE_ABSTRACT_STREAMABLE( exp, cls, ver ) \
    DECLARE_CASTABLE;                               \
    DECLARE_ABSTRACT_STREAMER( exp, cls, ver );      \
    DECLARE_STREAMABLE_OPS( cls );                   \
    DECLARE_STREAMABLE_CTOR( cls )                   \
```

\ // casting
\ // Streamer
\ // operadores
// constructor

Estas macros se pueden analizar en `<objstrm.h>` y su función está en declarar el constructor con parámetro *StreamableInit*, declarar una clase *Streamer* anidada en la clase que se desea hacer persistente y declarar los operadores de inserción/extracción:

```
opstream& operator<<( opstream&, const ClasePersistente& );
opstream& operator<<( opstream&, const ClasePersistente* );
ipstream& operator>>( ipstream&, ClasePersistente& );
ipstream& operator>>( ipstream&, ClasePersistente* );
```

```

// ografico.hpp

#include <objstrm.h>

class ObjetoGrafico : public TStreamableBase
{
    :

    // Attributes
protected :
        TRect          trBoundingBox;
        TColor         tcColor;
        BOOL           bActivo;
        BOOL           bSeleccionado;

        ControlPoint*   cp[ 4 ];

        // en cualquier parte, dentro de la declaración de la clase
        DECLARE_ABSTRACT_STREAMABLE( , ObjetoGrafico, 1 );
};

```

En el caso anterior se utiliza la macro `DECLARE_ABSTRACT_STREAMABLE` ya que `ObjetoGráfico` es una clase abstracta. El primer parámetro de esta macro sólo es relevante cuando la clase va a ser incluida en una DLL (*Dynamic Link Library*). El segundo parámetro debe ser el nombre de la clase y el tercero es la versión de la misma. Una de las ventajas que presenta el manejo de persistencia en Borland C++ 4.0 respecto a la versión 3.1, está en la posibilidad de verificar la versión de la clase a la cual pertenece un objeto en disco, y de esta manera poder leerlo adecuadamente.

Una vez incluida esta macro en la declaración de la clase, se debe agregar la macro `IMPLEMENT_STREAMABLE` correspondiente y definir los métodos `Streamer::read()` y `Streamer::write()` de acuerdo a la clase.

```

// ografico.cpp

IMPLEMENT_ABSTRACT_STREAMABLE( ObjetoGrafico );

void*
ObjetoGrafico::Streamer::Read( ipstream& is, uint32 ) const
{
    ObjetoGrafico* pO = GetObject();

    is >> pO->trBoundingBox;
    COLORREF c; is >> c; pO->tcColor = c;
}

```

```

        is >> pO->bActivo;
        is >> pO->bSeleccionado;

        pO->createControlPoints();

        return pO;
    }        // ObjetoGrafico::Streamer::Read

void
ObjetoGrafico::Streamer::Write( ostream& os ) const
{
    ObjetoGrafico* pO = GetObject();

    os << pO->trBoundingBox;
    COLORREF c = pO->tcColor;
    os << c;
    os << pO->bActivo;
    os << pO->bSeleccionado;

}        // ObjetoGrafico::Streamer::Write

```

La macro `IMPLEMENT_ABSTRACT_STREAMABLE` expande en las siguientes macros que también se encuentran en el archivo `<objstrm.h>`:

```

#define IMPLEMENT_ABSTRACT_STREAMABLE( cls )      \
    IMPLEMENT_STREAMER( cls );                    \
    IMPLEMENT_STREAMABLE_CTOR( cls );             \
    IMPLEMENT_STREAMABLE_POINTER( cls )

```

Cuando se trata de una clase concreta, como en el caso de la clase `Texto`, se incluye la macro `DECLARE_STREAMABLE` en la declaración de la clase, la cual a su vez expande las siguientes macros:

```

#define DECLARE_STREAMABLE( exp, cls, ver )
    DECLARE_CASTABLE;
    DECLARE_STREAMER( exp, cls, ver );           // clase Streamer
    DECLARE_STREAMABLE_OPS( cls );               // operadores
    DECLARE_STREAMABLE_CTOR( cls )               // constructor

```

```
// texto.hpp

class Texto : public ObjetoGrafico
{
    :
// Attributes
private :
    TFont*                ptfFont;
    TextTBuffer            transferBuffer;
    TChooseFontDialog::TData fontData;

    // persistencia
    DECLARE_STREAMABLE( , Texto, 1 );
};
```

En el caso de la clase Texto, se utiliza la macro IMPLEMENT_STREAMABLE1(Texto, ObjetoGrafico) para indicar que es descendiente de ObjetoGráfico, que a su vez es una clase persistente.

```
// texto.cpp

IMPLEMENT_STREAMABLE1( Texto, ObjetoGrafico );

void*
Texto::Streamer::Read( ipstream& is, uint32 ) const
{
    Texto* pO = GetObject();
    ReadBaseObject( STATIC_CAST( ObjetoGrafico*, pO ), is );

    is.readString( pO->transferBuffer.szText, MAX_TEXT );
    pO->transferBuffer.bFont = 0;

    is >> pO->fontData.Flags;
    :
    is.readString( pO->fontData.LogFont.lfFaceName, LF_FACESIZE );

    pO->ptfFont = new TFont( &pO->fontData.LogFont );

    return pO;
} // Texto::Streamer::Read

void
Texto::Streamer::Write( opstream& os ) const
```

```

{
    Texto* pO = GetObject();

    WriteBaseObject( STATIC_CAST( ObjetoGrafico*, pO ), os );

    os.writeString( pO->transferBuffer.szText );
    os << pO->fontData.Flags;
    :
    os.writeString( pO->fontData.LogFont.lfFaceName );

} // Texto::Streamer::Write

```

Para leer un objeto Texto, primero se debe construir y leer un objeto ObjetoGráfico, y de forma similar, para escribir un objeto Texto primero se deben escribir los componentes de ObjetoGráfico. Para esto se hace uso de dos funciones miembro de la clase *Streamer*: *ReadBaseObject()* y *WriteBaseObject()*, como se muestra en el ejemplo anterior.

```

// ogeometr.hpp

class ObjetoGeometrico : public ObjetoGrafico
{
    :
// Attributes

protected :
    BYTE          ucLineWidth;
    BYTE          ucLineStyle;

    // persistencia
    DECLARE_ABSTRACT_STREAMABLE( , ObjetoGeometrico, 1 );
};

// ogeometr.cpp

IMPLEMENT_ABSTRACT_STREAMABLE1( ObjetoGeometrico, ObjetoGrafico );

void*
ObjetoGeometrico::Streamer::Read( ipstream& is, uint32 ) const
{
    ObjetoGeometrico* pO = GetObject();
    ReadBaseObject( STATIC_CAST( ObjetoGrafico*, pO ), is );

    is >> pO->ucLineWidth;
    is >> pO->ucLineStyle;
}

```

```

        return pO;

    }    // ObjetoGeometrico::Streamer::Read

```

```

void
ObjetoGeometrico::Streamer::Write( ostream& os ) const
{
    ObjetoGeometrico* pO = GetObject();
    WriteBaseObject( STATIC_CAST( ObjetoGrafico*, pO ), os );

    os << pO->ucLineWidth;
    os << pO->ucLineStyle;

}    // ObjetoGeometrico::Streamer::Write

// linea.hpp

class Linea : public ObjetoGeometrico
{
    :

private :

    // persistencia
    DECLARE_STREAMABLE_FROM_BASE( , Linea, ObjetoGeometrico );
};

// linea.cpp

IMPLEMENT_STREAMABLE_FROM_BASE( Linea, ObjetoGeometrico );

```

En el ejemplo anterior se presenta el caso de una clase (Línea) que descende de una clase persistente (ObjetoGeométrico) y que no agrega nuevos atributos. En este caso no es necesario que Línea defina los métodos *Streamer::Read()* y *Streamer::Write()*, ya que si así lo hiciera estos métodos sólo evocarían a sus correspondientes en ObjetoGeométrico. Cuando se da esta situación se utiliza las macros `DECLARE_STREAMABLE_FROM_BASE` en la declaración de la clase e `IMPLEMENT_STREAMABLE_FROM_BASE` en la implementación. Este caso se repite en las clases Rectángulo y Elipse, descendientes de ObjetoCerrado.

Una vez que los ObjetosGráficos se han hecho persistentes, bajar el dibujo a disco o recuperarlo de disco es una tarea sencilla.

```
void
Dibujo::read( ifpstream& is )
{
    int count;
    ObjetoGrafico* pOG;

    is >> count;
    while ( count-- )
    {
        is >> pOG;
        dibujo->Add( pOG );
    }

} // Dibujo::read

void
Dibujo::write( ofpstream& os )
{
    os << dibujo->GetItemsInContainer();

    SetIterator it( *dibujo );
    while ( it )
    {
        os << it.Current();
        it++;
    }

} // Dibujo::write
```

Con el ejemplo de la jerarquía de clases ObjetosGráficos se cubren los casos posibles de persistencia de clases con herencia simple. Borland C++ 4.0 también provee la funcionalidad necesaria para la persistencia de clases con herencia múltiple virtual. En este caso las macros `DECLARE_STREAMABLE` e `IMPLEMENT_STREAMABLE` utilizadas son las mismas, y en los métodos `Streamer::Read()` y `Streamer::Write()` en vez de utilizar `ReadBaseObject()` y `WriteBaseObject()` como hemos visto, se utilizan `ReadVirtualBase()` y `WriteVirtualBase()` respectivamente.

9. Programación bajo MS-Windows

La generación de aplicaciones a ser ejecutadas en Windows involucra el manejo de dos nuevas técnicas¹: utilización de recursos y programación orientada a eventos. Una de las grandes ventajas para el usuario (y como veremos también para el programador) de trabajar en un ambiente operativo con una interfaz consistente, radica en que ya no es necesario que pase largas horas aprendiendo a manejar un programa. Se aprende a manejar el ambiente de trabajo (acciones de mouse, significado de los íconos, menús, uso del clipboard, etc.) y con eso ya se tiene el manejo básico de todas las aplicaciones en ese ambiente. Para el programador la ventaja está en que el sistema operativo es el que provee el soporte para el manejo de los elementos de interfaz. Este soporte está dado a través de **recursos**.

9.1. Recursos

Dentro de los recursos que un programa puede utilizar están los cursores de mouse, menús pull-downs y menús pop-up, tablas de aceleradores, bitmaps e íconos, cajas de diálogo, tablas de cadenas de caracteres, controles e inclusive fuentes propias. Estos recursos se pueden describir en un archivo ASCII con extensión .rc que luego es compilado (*Resource Compiler*) y encadenado a los módulos objeto para generar el programa ejecutable. A continuación se muestra una versión editada del archivo de recursos del editor gráfico de ejemplo.

```
// eg.rc

#include "eg.rh"

EgMenu MENU
{
    POPUP "&Archivos"
    {
        MENUITEM "&Nuevo", CM_NUEVO
        MENUITEM "&Cargar dibujo...", CM_CARGAR
        :
        MENUITEM SEPARATOR
        MENUITEM "&Terminar", CM_EXIT
    }

    POPUP "&Edición"
    {
        MENUITEM "&Deshacer\tAlt+BkSp", CM_DESHACER
        MENUITEM "&Rehacer\tShift+Alt+BkSp", CM_REHACER
        :
        MENUITEM SEPARATOR
        MENUITEM "&Seleccionar todo", CM_SELECC_TODO, GRAYED
    }
}
```

```

    }

    POPUP "Ay&uda"
    {
        MENUITEM "Como usar la &ayuda\tF1", CM_AYUDA_AYUDA
        MENUITEM "&Contenido", CM_CONTENIDO
        :
    }
}

About DIALOG 64, 75, 176, 68
STYLE WS_POPUP | WS_VISIBLE | WS_CAPTION
CLASS "bordlg"
CAPTION "Editor Gráfico"
FONT 8, "MS Sans Serif"
{
    CONTROL "", IDOK, "BorBtn", BS_DEFPUSHBUTTON | WS_CHILD | NOT
    WS_VISIBLE | WS_TABSTOP, 185, 70, 37, 25
    CONTROL "", IDCANCEL, "BorBtn", BS_PUSHBUTTON | WS_CHILD | NOT
    WS_VISIBLE | WS_TABSTOP, 186, 41, 37, 25
    CONTROL "", -1, "BorShade", BSS_GROUP | BSS_CAPTION | BSS_LEFT |
    WS_CHILD | WS_VISIBLE, -29, -6, 216, 80
    ICON 100, 100, 143, 6, 18, 20
    CONTROL "Button", 100, "BorBtn", BBS_BITMAP | WS_CHILD | WS_VISIBLE,
13, 6, 114, 55
    PUSHBUTTON "ok", 1, 138, 48, 28, 14
}

CURSOR_LINEA CURSOR
{
    '00 00 02 00 01 00 20 20 00 00 07 00 07 00 30 01'
    '00 00 16 00 00 00 28 00 00 00 20 00 00 00 40 00'
    '00 00 01 00 01 00 00 00 00 00 02 00 00 00 00'
    '00 00 00 00 00 00 00 00 00 00 00 00 00 00 00'
    '00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00'
    :
}

CURSOR_RECTANGULO CURSOR
{
    :
}

BMP_RECTANGULO BITMAP
{
    :
}

```

¹ También están las DLL o bibliotecas de funciones de encadenamiento dinámico, pero están fuera del alcance

```

BMP_ELIPSE BITMAP
{
    :
}
EgAccel ACCELERATORS
{
    VK_BACK, CM_DESHACER, VIRTKEY, ALT
    VK_F1, CM_AYUDA_AYUDA, VIRTKEY
    VK_BACK, CM_REHACER, VIRTKEY, ALT, SHIFT
    VK_DELETE, CM_BORRAR, VIRTKEY, SHIFT
    VK_INSERT, CM_COPIAR, VIRTKEY, CONTROL
    VK_INSERT, CM_PEGAR, VIRTKEY, SHIFT
}

TEXT0 DIALOG 20, 96, 260, 65
STYLE DS_SYSMODAL | WS_POPUP | WS_VISIBLE | WS_CAPTION |
WS_SYSMENU
CAPTION "Texto"
FONT 8, "MS Sans Serif"
{
    EDITTEXT DLG_EDIT, 7, 7, 190, 51, ES_MULTILINE | ES_AUTOVSCROLL |
    ES_AUTOHSCROLL | WS_BORDER | WS_TABSTOP
    CONTROL "&Fuente", DLG_FONT, "BorCheck", BS_AUTOCHECKBOX |
    WS_CHILD | WS_VISIBLE | WS_TABSTOP, 208, 9, 36, 10
    DEFPUSHBUTTON "OK", IDOK, 203, 44, 50, 14
    PUSHBUTTON "Cancel", IDCANCEL, 203, 28, 50, 14
    CONTROL "", -1, "BorShade", BSS_GROUP | BSS_CAPTION | BSS_LEFT |
    WS_CHILD | WS_VISIBLE, 203, 7, 50, 14
}

```

Sin embargo, como se trata de recursos gráficos, usualmente no se escribe un archivo de recursos como el anterior, sino que se utiliza un programa graficador/generador de recursos tal como el *Resource Workshop* que hace parte del ambiente integrado de Borland C++ 4.0.

Como se ve en la figura 8.1 la generación de un programa para Windows también requiere de un archivo de definición (.def) que provee información al encadenador acerca del contenido y requerimientos del módulo. El formato de este archivo se analiza a continuación:

NAME	Editor Gráfico	(opcional)
DESCRIPTION	'Ejemplo para curso de POO'	(opcional)
EXETYPE	WINDOWS	(obligatorio)

del contenido de este curso.

CODE	PRELOAD MOVEABLE	(obligatorio)
DATA	PRELOAD MOVEABLE MULTIPLE	(obligatorio)
HEAPSIZE	1024	(opcional)
STACKSIZE	5120	(*opcional)
EXPORTS	...	(opcional)
IMPORTS	...	(opcional)

NAME especifica el nombre del programa y debe coincidir con el nombre del ejecutable. Si se está construyendo un DLL (*Dynamic Link Library*) en vez de un programa, se debe colocar LIBRARY.

DESCRIPTION permite especificar una descripción del programa o biblioteca de funciones.

EXETYPE determina el sistema operativo para el cual se construye el ejecutable.

CODE describe los atributos del segmento de código del programa. PRELOAD indica al encadenador que esta porción de la imagen del programa debe ser cargada en memoria al iniciar el programa. MOVEABLE indica a Windows que este módulo puede ser reubicado en memoria.

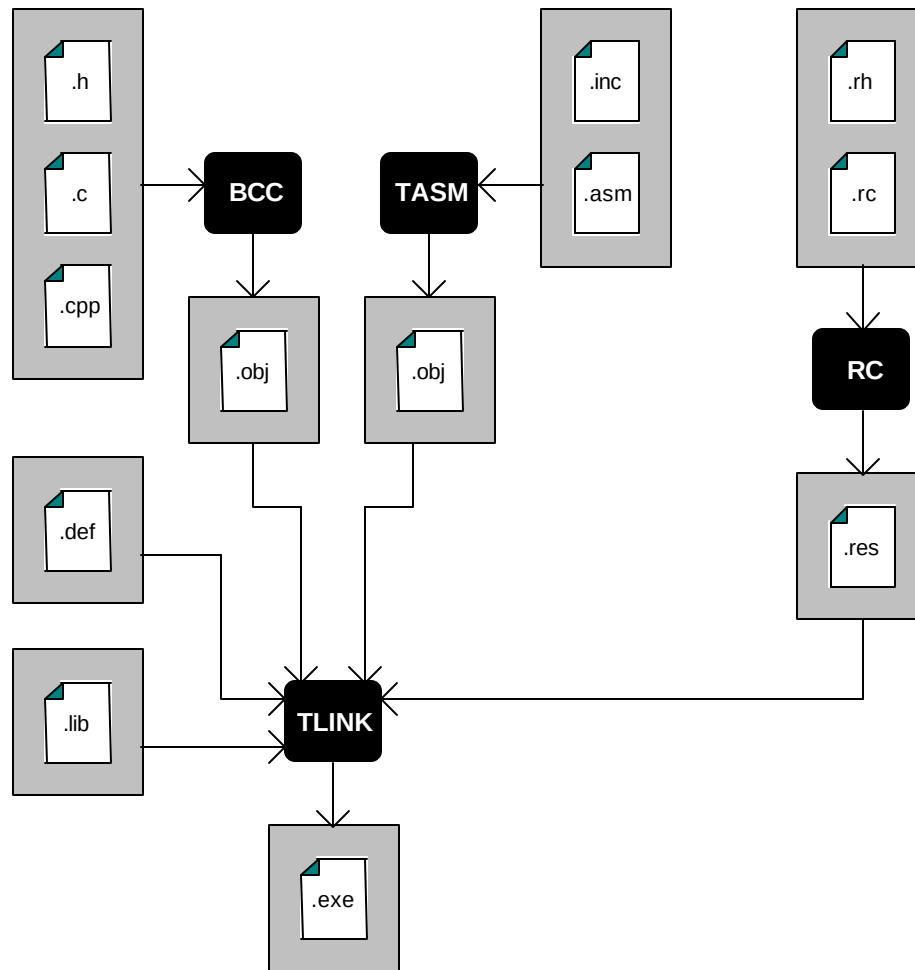


Fig. 8.1. Generación de un programa para MS-Windows.

DATA especifica los atributos de los segmentos de datos. La opción MULTIPLE asegura que cada instancia de la aplicación tiene sus propios segmentos de datos.

HEAPSIZE especifica el tamaño del montón (*heap*) local del programa.

STACKSIZE especifica el tamaño de la pila (*stack*) local del programa. Esta opción **no** puede ser especificada si se está construyendo un DLL.

EXPORTS indica el conjunto de funciones del programa o DLL que pueden ser llamadas por otros programas o por Windows. Estas funciones son conocidas como *export functions*, *callback*, o *callback functions*.

IMPORTS indica el conjunto de funciones que el programa o DLL requiere de otros programas o del Windows API (*Application Program Interface*). Borland C++ encadena automáticamente IMPORT.LIB, IMPORT32.LIB o IMPRTW32.LIB, por lo que normalmente no es necesario incluir esta sección.

9.2. Programación orientada a eventos

En programación es muy común hacer llamadas al sistema operativo para que éste lleve a cabo alguna tarea en particular. Por ejemplo, en lenguaje C es usual utilizar *fopen(...)* para abrir un archivo. Esta función, de la biblioteca *<stdio.h>*, incluye las llamadas necesarias al sistema operativo para abrir el archivo.

La programación en Windows implica un enfoque diferente. Si bien provee más de 550 funciones² que pueden ser llamadas desde el programa, Windows **también hace llamadas al programa**, y lo hace a través del envío de mensajes. Todos estos mensajes comienzan con el prefijo WM_ para indicar que se trata de un mensaje de Windows. Windows mantiene una *cola de mensajes* para cada aplicación en ejecución. Cada vez que sucede un evento, lo traduce en un mensaje y lo coloca en la cola de mensajes de la aplicación. El programa va sacando mensajes de esta cola en un ciclo conocido como "message loop":

```
while ( GetMessage( &msg, NULL, 0, 0 ) )    // función del Windows API
{
    TranslateMessage( &msg );                // función del Windows API
    DispatchMessage( &msg );                 // función del Windows API
}
```

La variable *msg* es una estructura de tipo *MSG*, definida en *<windows.h>* de la siguiente forma:

```
typedef struct tagMSG
{
    HWND      hwnd;        // ventana destino
    WORD      message;     // mensaje
    WORD      wParam;      // dependiente del mensaje
    LONG      lParam;      // dependiente del mensaje
    DWORD     time;        // ocurrencia del evento
    POINT     pt;          // coordenadas del evento
} MSG;
```

```
// donde

typedef struct tagPOINT
{
    int      x;
    int      y;
}    POINT;
```

La función *GetMessage(&msg, NULL, 0, 0)* inicia el ciclo sacando un mensaje de la cola y dejándolo en *msg*. El segundo, tercero y cuarto parámetros son dejados en nulo para indicar a Windows que se desea recibir todos los mensajes, para cualquiera de las ventanas creadas por la aplicación. Si el campo *message* del mensaje recibido en *msg* es diferente de *WM_QUIT*, *GetMessage()* retorna un valor distinto de cero. *WM_QUIT* hace que el ciclo termine y con esto termina el programa.

La llamada a la función *TranslateMessage(&msg)* retorna el mensaje a Windows para su traducción según el manejador de teclado instalado (español, inglés, etc.).

La llamada a la función *DispatchMessage(&msg)* también retorna el mensaje a Windows para que éste lo despache a la ventana apropiada. Toda ventana tiene entonces una rutina para el procesamiento de mensajes, usualmente llamada "*window proc*", con un esquema como el siguiente:

```
long
FAR PASCAL
WndProc( HWND hwnd, WORD message, WORD wParam, LONG lParam )
{
    :
    switch ( message )
    {
        case WM_CREATE:    hdc = GetDC( hwnd );
                           :
                           return 0;

        case WM_PAINT:     hdc = BeginPaint( hwnd, &ps );
                           :
                           EndPaint( hwnd, &ps );
                           return 0;
```

² Windows API (Windows Application Program Interface)

```

        case WM_KEYDOWN: switch ( wParam )
                           case VK_LEFT:      xCaret = ...;
                           break;

                           case VK_RIGHT:     xCaret = ...;
                           break;

                           :
                        }

        SetCaretPos( xCaret, ... );
        return 0;

    case WM_DESTROY: PostQuitMessage( 0 );
                    return 0;

    }

    return DefWindowProc( hwnd, message, wParam, lParam );
}

```

A excepción de los manejadores de dispositivos (*device drivers*) que deben procesar interrupciones, Windows trata todo programa como si fuera el único en ejecución. Windows no conmuta la ejecución de un programa a otro en forma arbitraria, sino cuando el programa en ejecución llama a la función *GetMessage()*. En ese momento, si el programa no tiene ningún mensaje en cola, y Windows encuentra que hay otro que sí tiene, entonces conmuta la ejecución a este último. De aquí que se diga que Windows tiene un comportamiento de multitarea "a saltos" (*jumpy multitasking*). Cuando un programa tiene un proceso largo por realizar, los demás programas quedan suspendidos. A diferencia de otros sistemas multitarea tradicionales, donde hay un reloj (usualmente en hardware) que marca las tajadas de tiempo que son asignadas a cada tarea, Windows provee multitarea en la medida que los programas llamen a la función *GetMessage()*. Este comportamiento se conoce como *nonpreemptive multitasking*.

Como Windows puede enviar cerca de 130 mensajes diferentes a una aplicación, esta función *WndProc()* tiende a volverse enorme. Para solucionar esto, Borland sigue el estándar planteado por AT&T (también seguido por Microsoft y otros), donde se utilizan "tablas de respuesta" (*response tables*) que permiten asociar cada método con el mensaje (o mensajes) al que da respuesta. Este enfoque es nuevo en ObjectWindows³ 2.0 y completamente diferente al utilizado

³ ObjectWindows Library (OWL) es la biblioteca de clases que Borland provee para el manejo de los servicios del Windows API.

en ObjectWindows 1.0 (Borland C++ 3.1), que era estándar, sino una extensión del compilador de Borland.

9.3. Tablas de respuesta (*response tables*)

Para hacer uso de las tablas de respuesta se deben llevar a cabo los cuatro pasos siguientes:

1. Declarar la tabla de respuestas
2. Definir la tabla de respuestas
3. Definir las entradas de la tabla de respuestas
4. Definir los métodos que aparecen en la tabla de respuestas

La declaración de la tabla de respuestas de una clase se hace dentro de la declaración de la clase y para esto se utiliza la macro `DECLARE_RESPONSE_TABLE` que encapsula los detalles sintácticos y que recibe como argumento el nombre de la clase:

```
// vedicion.hpp

class VentanaEdicion : public TWindow
{
    :

    // Attributes
    private :
        Dibujo*      pDibujo;
        Tool*        pTools[ NUMTOOLS ];
        ToolWindow*  pToolWindow;
        TPrinter*    pPrinter;

        typedef TStackAsVector<UserEvent> Stack;
        Stack*      undoStack;
        Stack*      redoStack;

        :

        DECLARE_RESPONSE_TABLE( VentanaEdicion );
};
```

Una vez declarada, se debe definir la tabla de respuestas y esto se hace por fuera de la declaración de la clase, usualmente en el archivo de implementación de la misma. Se utiliza en este caso la macro `DEFINE_RESPONSE_TABLEx`, donde *x* representa el número de clases de las cuales descende (directamente, esto es, superclases inmediatas) la clase para la cual se está definiendo la tabla de respuestas:

```

// vedicion.cpp

#include "rg.rh"                                // constantes CM_
DEFINE_RESPONSE_TABLE1( VentanaEdicion, TWindow )

    EV_WM_LBUTTONDOWN,                          // mensajes de Windows
    EV_WM_LBUTTONUP,
    EV_WM_MOUSEMOVE,
    EV_WM_LBUTTONDOWNBLCLK,
    EV_WM_SIZE,
    EV_WM_DROPFILES,

    EV_COMMAND( CM_NUEVO, CmNuevo ),            // siempre activo
    EV_COMMAND( CM_CARGAR, CmRecuperar ),        // siempre activo

    EV_COMMAND( CM_ALMACENAR, CmAlmacenar ),     // depende del estado
    EV_COMMAND_ENABLE( CM_ALMACENAR, CeAlmacenar ), // del dibujo

    :

    EV_COMMAND_AND_ID( CM_TOOL+0, CmTool ),      // mensaje + wParam
    EV_COMMAND_AND_ID( CM_TOOL+1, CmTool ),
    EV_COMMAND_AND_ID( CM_TOOL+2, CmTool ),
    EV_COMMAND_AND_ID( CM_TOOL+3, CmTool ),
    EV_COMMAND_AND_ID( CM_TOOL+4, CmTool ),
    EV_COMMAND_AND_ID( CM_LINE+1, CmLine ),
    EV_COMMAND_AND_ID( CM_LINE+3, CmLine ),
    EV_COMMAND_AND_ID( CM_LINE+5, CmLine ),
    EV_COMMAND_AND_ID( CM_LINE+7, CmLine ),
    EV_COMMAND_AND_ID( CM_LINE+9, CmLine ),
    EV_COMMAND_AND_ID( CM_STYLE+0, CmStyle ),
    EV_COMMAND_AND_ID( CM_STYLE+2, CmStyle ),

END_RESPONSE_TABLE;

```

La definición de la tabla de respuestas termina siempre con la macro `END_RESPONSE_TABLE`.

Las entradas de la tabla de respuestas asocian un evento de Windows con una función miembro en particular. Cuando una ventana o control recibe un mensaje, revisa la lista de entradas de su tabla de respuestas en busca de un método para ese evento. Si lo encuentra, le pasa el mensaje para su procesamiento. Si no lo encuentra, pasa el mensaje a la ventana que la contiene (*parent window*). Si se trata de la ventana principal, entonces pasa el mensaje al objeto aplicación (de la

clase *TApplication*) del programa. Si el objeto aplicación no tiene una entrada para dar respuesta al mensaje, entonces es procesado por *ObjectWindows* que normalmente lo pasa a *Windows* por medio de la función *DefWindowProc()* del API. *ObjectWindows* provee un gran número de macros para generar las entradas de las tablas de respuestas, y estas incluyen:

- Macros para respuesta a comandos de menú
- Macros para respuesta a mensajes estándar de *Windows*
- Macros para respuesta a mensajes de controles contenidos (*child windows/controls*)
- Macros para respuesta a mensajes del manejador de documentos (*document manager*)
- Macros para respuesta a mensajes controles *VBX* (*Visual Basic Controls*)

ObjectWindows provee un gran número de macros que generan las entradas necesarias para responder a eventos de menú, así como entradas que permiten la activación/desactivación de los comandos del menú. En las tablas siguientes se presentan las macros y el prototipo de la función miembro correspondiente:

Macro y Prototipo	Descripción
<code>EV_COMMAND(CMD, método)</code> <code>void método();</code>	Llama a la función miembro <i>método()</i> cuando se recibe el comando <i>CMD</i> .
<code>EV_COMMAND_AND_ID(CMD, método)</code> <code>void método(WPARAM);</code>	Llama a la función miembro <i>método()</i> cuando se recibe el comando <i>CMD</i> . Pasa el identificador del mensaje (<i>WPARAM</i>) al método.
<code>EV_COMMAND_ENABLE(CMD, método)</code> <code>void método(TCommandEnabler&);</code>	Utilizada para especificar el método que ha de activar/desactivar una entrada de menú, un botón en el speed-bar y similares. El método es llamado automáticamente al abrirse el menú, o al obtener el foco el speed-bar.

```
// vedicion.cpp

#include "rg.rh"                                // constantes CM_
DEFINE_RESPONSE_TABLE1( VentanaEdicion, TWindow )
:
    EV_COMMAND( CM_NUEVO, CmNuevo ),           // siempre activo
    EV_COMMAND( CM_CARGAR, CmRecuperar ),       // siempre activo
    :
    EV_COMMAND( CM_ALMACENAR, CmAlmacenar ),    // depende del estado
    EV_COMMAND_ENABLE( CM_ALMACENAR, CeAlmacenar ), // del dibujo
    :
    EV_COMMAND_AND_ID( CM_TOOL+0, CmTool ),     // mensaje + wParam
    EV_COMMAND_AND_ID( CM_TOOL+1, CmTool ),
    :
```

```

END_RESPONSE_TABLE;-----
void
VentanaEdicion::CmNuevo()
{
    if ( okToClose() )
    {

        pDibujo->flush();

        undoStack->Flush();
        redoStack->Flush();
        :
    }

}    // VentanaEdicion::CmNuevo

// -----
void
VentanaEdicion::CmAlmacenar()
{
    almacenar();

}    // VentanaEdicion::CmAlmacenar

// -----
void
VentanaEdicion::CeAlmacenar( TCommandEnabler& ce )
{
    ce.Enable( pDibujo->dibujosIsEmpty() ? FALSE : TRUE );

}    // VentanaEdicion::CeAlmacenar

// -----
void
VentanaEdicion::CmTool( WPARAM wToolWin )
{
    ucCurTool = wToolWin - CM_TOOL;
    :

}    // VentanaEdicion::CmTool

```

OWL 2.0 también provee macros para aquellos mensajes que se desean procesar tal cual han sido generados:

Macro y Prototipo	Descripción

EV_MESSAGE(MSG, método) LRESULT método(WPARAM, LPARAM);	Llama a la función miembro <i>método()</i> cuando se recibe el mensaje <i>definido por el usuario MSG</i> . MSG es pasado al método sin modificación.
EV_REGISTERED(MSG, método) LRESULT método(WPARAM, LPARAM);	Llama a la función miembro <i>método()</i> cuando se recibe el mensaje <i>registrado MSG</i> . MSG es pasado al método sin modificación.

OWL provee las macros necesarias para todos los mensajes estándar de Windows. El nombre de la macro predefinida se encuentra agregando el prefijo EV_ al nombre del mensaje. El nombre del método asociado se encuentra quitando WM_ del nombre del mensaje, agregando el prefijo *Ev*, y pasando a mayúsculas /minúsculas el resto:

Macro	Descripción
EV_WM_PAINT	Llama a la función miembro <i>EvPaint()</i> cuando se recibe el mensaje WM_PAINT.
EV_WM_LBUTTONDOWN	Llama a la función miembro <i>EvLButtonDown()</i> cuando se recibe el mensaje WM_LBUTTONDOWN.
etc.	<i>ObjectWindows Reference Guide, Capítulo 2.</i>

Las macros para respuesta a mensajes de notificación de ventanas contenidas (*child ID notification messages*) pueden ser manejados de formas diferentes por medio de diferentes macros. Estas macros permiten procesar notificaciones de diferentes ventanas hijas en un sólo método, pasar todos los códigos de notificación de un control a uno de los métodos de la ventana, o procesar los códigos de notificación de un control en un método del mismo control:

Macro y Prototipo	Descripción
EV_CHILD_NOTIFY(ID, código, método) void método();	Utilizada cuando se desean procesar las notificaciones de uno o más controles (botón, lista, etc.) en un método de la ventana padre.
EV_CHILD_NOTIFY_AND_CODE(ID, código, método) void método(WPARAM código);	Envía el mensaje ID, con código de notificación código, al método <i>método()</i> .
EV_CHILD_NOTIFY_ALL_CODES(ID, método) void método(WPARAM código);	Envía el mensaje ID al método <i>método()</i> , sin importar código de notificación.
EV_CHILD_NOTIFY_AT_CHILD(código, método) void método();	Envía el código de notificación código, al método <i>método()</i> del mismo control.

```

// textdlg.hpp

class TextDialog : public TDialog
{
    :

// Attributes
private :
    TChooseFontDialog::TData* pFontData;

    TEdit*      pEdit;
    TCheckBox*  pFont;

    DECLARE_RESPONSE_TABLE( TextDialog );
};

// textdlg.cpp

DEFINE_RESPONSE_TABLE1( TextDialog, TDialog )

    EV_CHILD_NOTIFY( DLG_FONT, BN_CLICKED, handleFont ),

END_RESPONSE_TABLE;

//-----
void
TextDialog::handleFont()
{
    TChooseFontDialog( this, *pFontData ).Execute();

    pFont->SetCheck( BF_UNCHECKED );
    pEdit->SetFocus();

} // TextDialog::handleFont

```

A continuación se presenta el código completo del programa de ejemplo, donde se pueden apreciar los diferentes tópicos analizados en el transcurso de este documento.

Referencias

- [Coad] Coad, P., Yourdon, E., *Análise Baseada em Objetos*. Editora Campus, Rio de Janeiro, 1992.
- [Jackson] Jackson, M. A., *Principles of Program Design*. Academic Press, London, 1975.
- [Jackson] Jackson, M. A., *System Development*. Prentice-Hall, London, 1983.
- [Meyer] Meyer, B., *Object-Oriented Software Construction*. Prentice-Hall, Hemel Hempstead, 1988.
- [Sommerville] Sommerville, I., *Software Engineering*. Addison-Wesley, Wokingham, 1989.
- [Wirfs-Brock] Wirfs-Brock, R., Wilkerson, B. y Wiener, L., *Designing Object-Oriented Software*, Prentice Hall, Englewood Cliffs, 1990.